

СОДЕРЖАНИЕ

Введение	4
1. Описание функционала системы. Диаграмма вариантов использования	5
2. Диаграмма классов анализа	15
3. Диаграмма последовательности	27
4. Диаграмма классов уровня проектирования (реализации)	47
5. Диаграмма деятельности	61
6. Диаграммы компонентов	71
7. Диаграммы развертывания	79
Список источников и литературы	87

Введение

Использование практикума в учебном процессе нацелено на решение следующих основных задач:

- углубление знаний по курсу «Анализ и концептуальное моделирование систем»;
- фиксация и конкретизация на этой основе учебных целей и задач дисциплины;
- рациональное распределение учебного времени по разделам курса и видам учебных занятий;
- определение круга источников, учебной, методической и научной литературы, необходимых для освоения дисциплины.

Практикум содержит материал по практическим работам, включающим формирование моделей анализа, проектирования и реализации. Приведены основные этапы, правила и примеры построения диаграмм UML.

1. Описание функционала системы. Диаграмма вариантов использования

Цель работы - изучить структуру и функционал рассматриваемой информационной системы, освоить правила построения диаграммы вариантов использования.

Задачи:

- изучить предметную область по заданным вариантам;
- определить на концептуальном уровне состав элементов системы;
- описать функции рассматриваемой системы с помощью диаграммы вариантов использования.

Используемое ПО: Visual Paradigm, Draw.io, Rational Rose.

Краткая теория

На сегодняшний день существует два вида разработки программных систем: разработка под заказ; типовой тиражируемый продукт на продажу. И в том и в другом случае, для обоснования необходимости разработки программной системы обязательно нужно провести исследование предметной области.

В первом случае – в случае разработки программной системы под заказ (костомизация, адаптация, модернизация существующей системы) необходимо в качестве предметной области рассмотреть компанию-заказчик. Обычно заказчик говорит, что именно ему необходимо, но для реализации его целей и обоснования необходимости разработки именно этого продукта, стоит изучить их бизнес-процессы, выявить наиболее «узкие» места, то есть проблемы и сделать предложение по их решению. То есть объектом исследования выступает как конкретная организация заказчика.

Во втором случае - случае разработки типовых программ необходимо изучить рынок похожих программных продуктов. Данную процедуру выполняет как отдел маркетинга, так и аналитик компании разработчика – создают концепцию продукта. То есть объектом исследования выступает рынок похожих программных средств. Основной целью является разработка такой системы или ее ключевых элементов, которые бы стали поводом для покупки именно этого продукта.

Диаграмма вариантов использования (англ. use-case diagram) – диаграмма, описывающая, какой функционал разрабатываемой программной системы доступен каждой группе пользователей. Диаграммы вариантов использования

описывают функциональное назначение системы или то, что система должна делать.

Разработка диаграммы преследует *следующие цели*:

- определить общие границы и контекст моделируемой предметной области;
- сформулировать общие требования к функциональному поведению проектируемой системы;
- разработать исходную концептуальную модель системы для ее последующей детализации в форме логических и физических моделей;
- подготовить исходную документацию для взаимодействия разработчиков системы с ее заказчиками и пользователями.

Диаграмма вариантов использования состоит из актеров, для которых система производит действие, и собственно действия Use Case, которое описывает то, что актер хочет получить от системы. Дополнительно в диаграммы могут быть добавлены комментарии.

Между актерами и вариантами использования могут быть различные виды взаимодействия (рис. 1 а). Основные виды взаимодействия:

Простая ассоциация - отражается линией между актером и вариантом использования (без стрелки). Отражает связь актера и варианта использования.

Направленная ассоциация - то же что и простая ассоциация, но показывает, что вариант использования инициализируется актером. Обозначается стрелкой.

Наследование - показывает, что потомок наследует атрибуты и поведение своего прямого предка. Может применяться как для актеров, так для вариантов использования.

Расширение (extend) - показывает, что вариант использования расширяет базовую последовательность действий и вставляет собственную последовательность. При этом в отличие от типа отношений "включение" расширенная последовательность может осуществляться в зависимости от определенных условий.

Включение (include) - показывает, что вариант использования включается в базовую последовательность и выполняется всегда.

Актером (действующим лицом, актантом, актором) называется любой объект, субъект или система, взаимодействующая с моделируемой системой извне. В свою очередь **вариант использования** – это спецификация сервисов (функций), которые система предоставляет актеру.

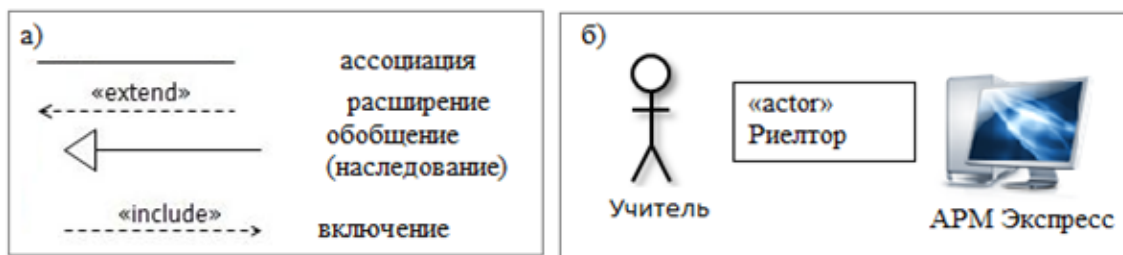


Рисунок 1. Элементы диаграммы вариантов использования:

а) отношения; б) актеры

Актера графически можно отобразить тремя способами (рис. 1 б): «проволочный человек», класс с текстовым стереотипом «actor», произвольная иконка.

Способ отображения в виде «проволочного человечка» является самым распространенным.

Вариант использования обозначается на диаграмме эллипсом (рис. 2), внутри которого содержится его описание, обозначающее выполнение какой-либо операции или действия.



Рисунок 2. Примеры вариантов использования

Примечания предназначены для включения в диаграмму произвольной текстовой информации, имеющей непосредственное отношение к контексту разрабатываемой системы (рис. 3). Линия, соединяющая примечание и элемент диаграммы, называется якорем (фиксацией).



Рисунок 3. Пример примечания

Связи между актерами и вариантами отображаются с использованием **отношений** четырех видов:

- ассоциаций;
- обобщения;
- включения (зависимость со стереотипом «include»);
- расширения (зависимость со стереотипом «extend»).

Применительно к рассматриваемой диаграмме **отношение ассоциации** служит для обозначения взаимодействия актера с вариантом использования (рис. 4).

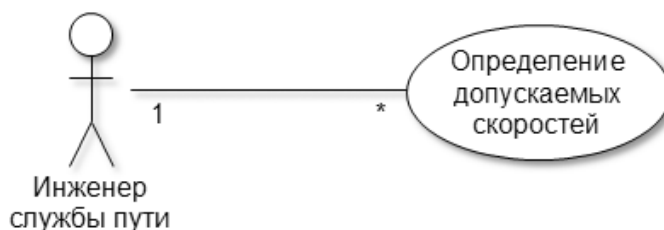


Рисунок 4. Пример ассоциации

Отношение обобщения служит для указания того факта, что некоторая сущность А может быть обобщена до сущности В. Графически данное отношение обозначается сплошной линией со стрелкой, в виде незакращенного треугольника, от потомка к родителю (рис. 5).

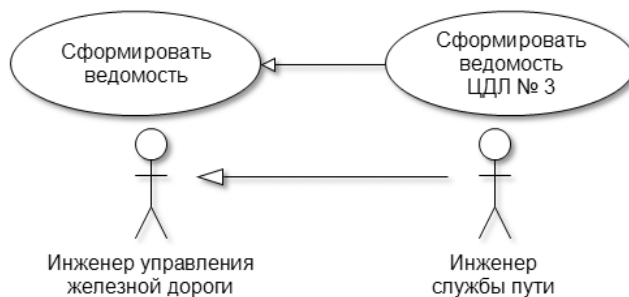


Рисунок 5. Примеры обобщения

Отношения включения и расширения являются частным случаем отношения **зависимости** и могут иметь место только между двумя вариантами использования. Они отображаются штриховой стрелкой с указанием стереотипа.

Отношение включения указывает, что некоторое заданное поведение одного варианта использования **обязательно** включается в качестве составного компонента в последовательность поведения другого варианта использования (рис. 6).

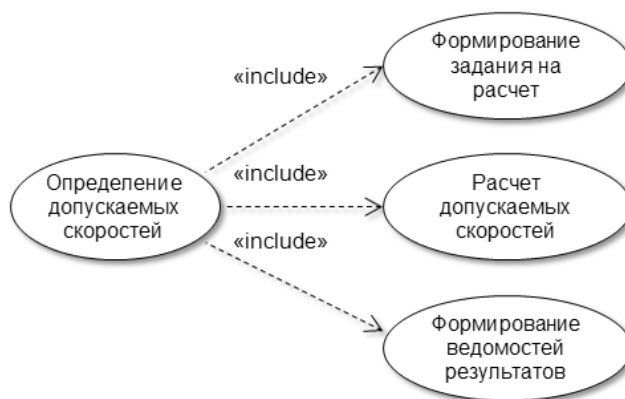


Рисунок 6. Пример включения

Стрелка расширения должна быть направлена от включаемого варианта к базовому варианту и помечена стереотипом «extend» (рис. 7).

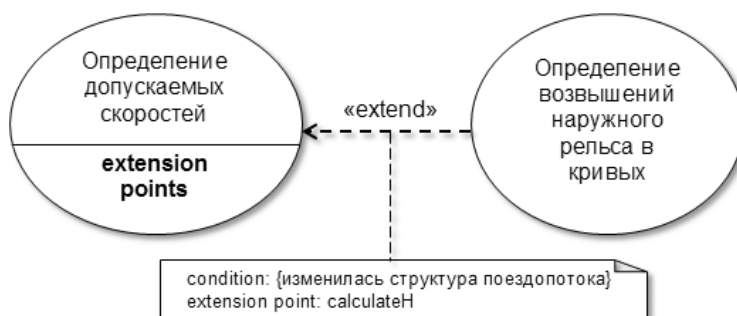


Рисунок 7. Пример расширения

Этапы реализации:

Шаг 1. Выбрать свой вариант учебного задания из предложенных вариантов.

Шаг 2. Проанализировать существующие на рынке на текущий момент решения и собрать предварительную информацию.

Шаг 3. На основе проведенного анализа сделать вывод о том, что можно было бы добавить, или исключить (какие звенья системы усовершенствовать).

Шаг 4. Выделить основные функции реализуемой системы в виде таблицы.

Шаг 5. Описать ожидаемые результаты реализации моделируемой системы.

Шаг 6. Спроектировать диаграмму вариантов использования. Проанализировать, какие активные субъекты должны взаимодействовать с будущей системой.

Шаг 7. Добавить в диаграмму актеров и соответствующие варианты использования.

Шаг 8. Расставить связи между элементами.

Пример анализа и построения диаграммы вариантов использования для системы организации продаж одежды и аксессуаров

Шаг 1. Выберем вариант - анализ системы организации по продаже одежды и аксессуаров.

Шаг 2. Проанализируем уже существующие сайты. Похожие сайты на рынке существуют уже давно, одни из них коммерческие, другие – нет и направлены на предоставление той или иной справочной информации. Например, сайт Lamoda, сайт Обновка своими руками.

Анализ сайта «Lamoda». Функционал системы богат: от поиска нужных товаров через строку «поиска» до разбиения товаров по самым разным категориям: мужчины, женщины и дети; одежда, обувь и аксессуары; новинки и бренды и другие. Также сайт предлагает пользователям зарегистрироваться и создать личный кабинет. Выявленные недостатки: *нет списка товаров, на производство которых можно оформить заказ, нет возможности отправки сообщений через социальные сети непосредственно через сайт, нет отдельной страницы с различной информацией по теме сайта.*

Анализ сайта «Обновка своими руками». Некоммерческий сайт, предоставляющий всевозможные варианты пошива и вязания одежды и соответствующую справочную информацию. Имеет свою карту и рекомендации литературы по его теме. Недостатки: *сайт давно не обновлялся, информация скудная, отсутствует отдельное поле регистрации, отсутствует поле для получения информации о стадии выполнения заказов, нет информации о клиентах.*

Шаг 3. На основе проведенного анализа сделаем вывод об основных необходимых функциях:

- создать список товаров как уже выставленных на продажу, так и на производство которых можно оформить заказ;
- создать поле регистрации личного кабинета для связи с производителем/продавцом;
- создать чаты, где можно оставить отзывы о работе или пообщаться на связанные с главной идеей сайта темы;
- предоставить возможность отправки сообщений через соц. сети непосредственно через сайт;
- создать поле для получения информации о стадии выполнения заказов.

- создать базу данных, хранящую информацию о клиентах (ФИО, почту, телефон и др.).
- создать поле с краткой информацией о сайте и его возможностях.
- создать страницы с различной информацией по теме сайта.

Шаг 4. Опишем решения в виде таблицы (табл. 1).

Таблица 1. Описание элементов

Наименование	Краткое описание
Список товаров	Создать список товаров как уже выставленных на продажу, так и на производство которых можно оформить заказ.
Поле регистрации	Создать поле регистрации личного кабинета для связи с производителем/продавцом.
Обратный отклик	Создать чаты, где можно оставить отзывы о работе или пообщаться на связанные с главной идеей сайта темы. Предоставить возможность отправки сообщений через соц. сети непосредственно через сайт.
Возможность просмотра истории	Создать поле для получения информации о стадии выполнения заказов.
Сохранение данных о пользователях	Создать базу данных, хранящую информацию о клиентах (ФИО, почту, телефон и др.).
Предоставление информации по теме сайта	Создать поле с краткой информацией о сайте и его возможностях. Создать страницы с различной информацией по теме сайта.

Шаг 5. Опишем ожидаемые результаты реализации моделируемой системы: увеличение количества привлеченных клиентов, увеличение прибыли, уменьшение времени поиска товара, уменьшение скорости предоставления нужной информации.

Шаг 6. Спроектируем диаграмму вариантов использования. Проанализируем, какие активные субъекты должны взаимодействовать с будущей системой.

Шаг 7. Добавим в диаграмму актеров и варианты использования.

Шаг 8. Расставим связи между элементами (рис. 8).

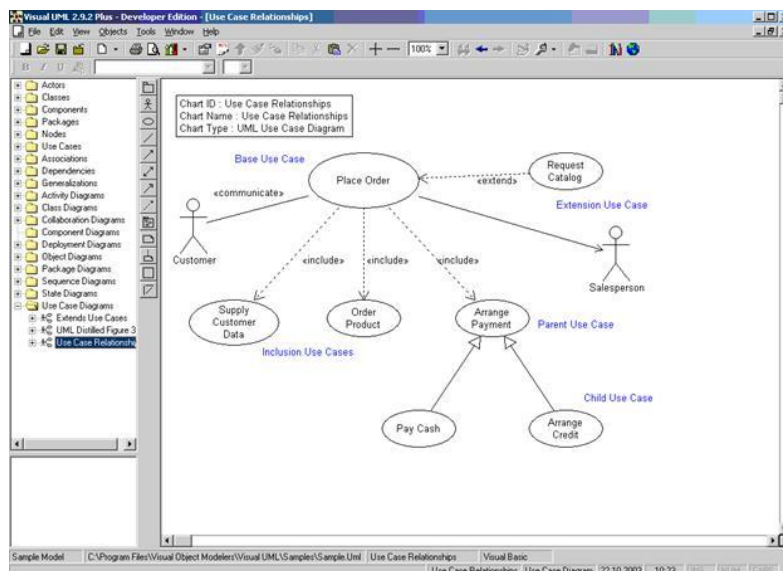


Рисунок 8. Пример диаграммы Use Case

Анализ результатов. Выполнен анализ предметной области, изучены существующие на рынке наиболее известные сайты по продаже одежды и аксессуаров. На основе проведенного анализа выявлены основные «узкие» места и выявлены возможности для их решения. Для удобства основные функции реализуемой системы описаны в виде таблицы. Создана диаграмма вариантов использования, определены актеры и основные прецеденты, установлены связи между ними.

Правила и рекомендации по разработке диаграмм вариантов использования

1. Рекомендуется вначале построить контекстную диаграмму, на которой отображаются основные варианты использования (функции) системы, а затем для каждого из них построить диаграммы декомпозиции (детализации).
2. Чрезмерная детализация вариантов использования не требуется. Следует помнить, что вариант использования – это относительно крупный блок функциональности системы. Для детализации в дальнейшем будут использоваться другие виды диаграмм, более подходящие для этой цели.
3. Для лучшего восприятия отдельная диаграмма (контекстная или декомпозиции) не должна быть перенасыщена элементами. Рекомендуется отображать на диаграмме не более 15 вариантов использования.
4. Располагать элементы следует так, чтобы была видна логическая последовательность выполнения вариантов использования и было минимум пересечений между отношениями.

5. Перед построением диаграммы необходимо задокументировать потоки событий в системе.

6. На диаграммах не следует отображать особенности реализации вариантов использования и внутренней организации системы, связанные со спецификой используемых программных и аппаратных средств.

8. Пользуйтесь специальными компьютерными программами для построения диаграмм. Это существенно упростит весь процесс моделирования.

Задания для самостоятельного выполнения:

Согласно индивидуальному варианту учебного проекта:

1. выполните анализ предметной области, выявите основные «узкие» места и возможности для их решения. Основные функции реализуемой системы опишите в виде таблицы.

2. постройте диаграмму вариантов использования, рассматриваемой системы, в любом из предлагаемых программных продуктов, следуя порядку, предложенному выше.

Контрольные вопросы

1. Для чего используется язык UML?
2. Какие диаграммы входят в состав языка UML?
3. В чем смысл варианта использования?
4. Каково назначение диаграмм вариантов использования.
5. Назовите основные свойства вариантов использования.
6. Назовите основные компоненты диаграмм вариантов использования.
7. Что такое «действующее лицо»?
8. Какую роль могут играть действующие лица по отношению к варианту использования?
9. Каким образом анализ внешних событий позволяет определить варианты использования системы?
10. Что позволяют получить диаграммы вариантов использования?
11. Что показывает диаграмма вариантов использования?
12. Какие элементы содержит диаграмма вариантов использования?
13. Что служит хорошим источником для идентификации вариантов использования?
14. Что такое актер?
15. Перечислите типы актеров и расскажите об их особенностях.
16. Какие отношения возможны между актерами?

Варианты учебных проектов:

1. Моделирование системы учета военной техники на полигоне.
2. Моделирование системы управления проектами строительной компании.
3. Моделирование системы автоматизации конвейерного производства автомобилей.
4. Моделирование системы учета программы лояльности.
5. Моделирование организации автоперевозок грузов.
6. Моделирование мониторинга состояния компьютерного класса.
7. Моделирование системы повышения квалификации сотрудников.
8. Моделирование системы учета средств индивидуальной мобильности.
9. Моделирование АРМ сотрудника регистратуры частной клиники.
10. Моделирование АРМ делопроизводителя учебного отдела образовательного учреждения.
11. Моделирование АРМ сотрудника компании по разработке мобильных приложений.
12. Моделирование АРМ инструктора автошколы.

2. Диаграмма классов анализа

Цель работы - научиться выявлять внутреннюю архитектуру (определять состав и структуру) системы на концептуальном уровне.

Задачи:

- определить на концептуальном уровне состав элементов системы в виде классов анализа;
- определить статическую структуру системы на логическом уровне, используя различные типы отношений;
- построить диаграмму, найти альтернативные варианты реализации системы (подсистемы).

Используемое ПО: Visual Paradigm, Draw.io, Rational Rose.

Краткая теория

В жизненном цикле разработки программного обеспечения диаграммы классов занимают центральное место. Однако нужно понимать, что при разработке программного обеспечения, как правило, диаграммы классов в зависимости от этапа разработки постепенно моделируются с трех разных точек зрения по мере продвижения по уровням детализации. Поэтому, принято выделять 3 уровня абстракции классов:

- аналитический уровень (концептуальный уровень);
- уровень проектирования (уровень спецификации);
- уровень реализации (имплементационный уровень).

Аналитический уровень - интерпретация диаграмм как описание вещей в реальном мире.

Уровень проектирования - интерпретация диаграмм как описание абстракций программного обеспечения или компонентов со спецификациями и интерфейсами, но без привязки к конкретной реализации.

Уровень реализации - интерпретация диаграмм как описание реализаций программного обеспечения на определенной технологии и языке.

В данной практической работе будем рассматривать пока только концептуальный уровень, поскольку остальные уровни предполагается рассмотреть на следующих практических работах. Концептуальный уровень необходим для разумного сопровождения анализа требований диаграммами. Диаграммы классов этого уровня принято называть *диаграммами классов анализа*.

Класс анализа – это укрупненная абстракция одного или более классов (подсистем в проекте системы), которая на концептуальном уровне (без

точного определения атрибутов и операций) описывает некоторый фрагмент системы.

Диаграмма классов анализа необходима как для выявления внутренней архитектуры (определения подсистем и основных классов) системы, так и для поиска альтернативных вариантов реализации системы (подсистемы). На диаграммах классов анализа целесообразно указать только наименования классов, атрибуты и операции обычно не указываются (хотя можно их только определить), отложив их исследование на более поздние стадии детализации.

Существует три вида (стереотипа) классов анализа:

- *граничный класс (boundary class)* - класс, который располагается на границе системы с внешней средой и непосредственно взаимодействует с актерами, но является составной частью системы. Граничные классы – есть абстракции форм, панелей, коммуникационных интерфейсов, интерфейсов периферийных устройств, диалоговых окон. На диаграмме граничный класс имеет стереотип <<boundary>>;

- *класс-сущность (entity class)* - пассивный класс, абстракции основных понятий предметной области, хранимых в табличном или ином виде. Класс-сущность только принимает сообщения от других классов. Информация о нем должна постоянно храниться и не уничтожаться, даже если выключается система, или прекращается работа моделируемой системы или завершается программа. На диаграмме класс-сущность имеет стереотип <<entity>>;

- *управляющий класс (control class)* - класс, который отвечает за координацию действий других классов. Управляющий класс обычно выполняет достаточно сложные вычисления, а также управляет безопасностью, транзакциями и т. п. Важно, чтобы у диаграммы классов был хотя бы один управляющий класс. На диаграмме управляющий класс имеет стереотип <<control>>.

Эти три стереотипа стандартизированы в UML и используются, чтобы помочь разработчикам различать действия различных классов. Классы анализа графически отображаются либо в виде классического (односекционного) прямоугольника класса с указанием соответствующих стереотипов во французских кавычках, либо в виде графических примитивов, соответствующих этим видам классов (рис. 9).

Классы анализа, как элементы системы, естественно связаны между собой различными отношениями, изображенными (рис. 10).



Рисунок 9. Графическое изображение классов анализа

Связи между классами анализа отображаются с использованием отношений пяти видов:

- ассоциаций;
- агрегаций;
- композиций;
- наследования;
- зависимостей.

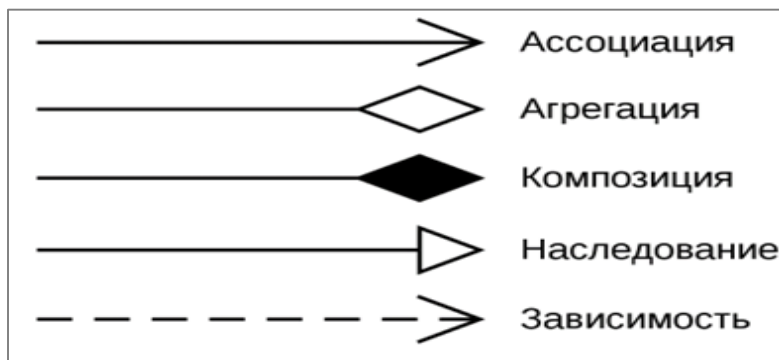


Рисунок 10. Виды отношений между классами анализа

Отношение ассоциации показывает, что объекты одного класса содержат информацию о существовании (наличии в памяти) объектов другого класса и между ними имеется некоторая логическая или семантическая связь (рис. 11).

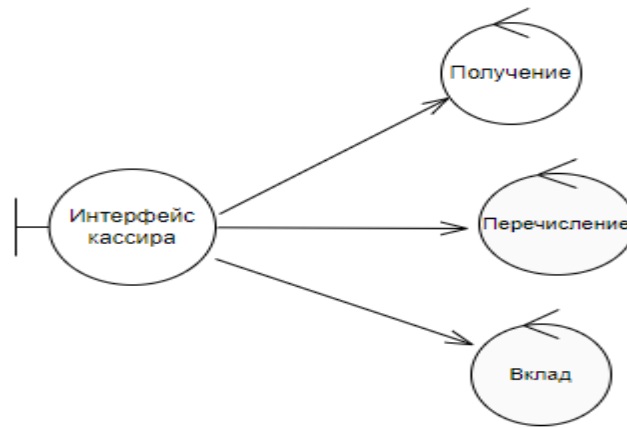


Рисунок 11. Отношение (направленной) ассоциации

Отношение агрегации указывает на отношение «часть»-«целое» и отображается сплошной линией с незакрашенным ромбиком со стороны «целого» (рис. 12).

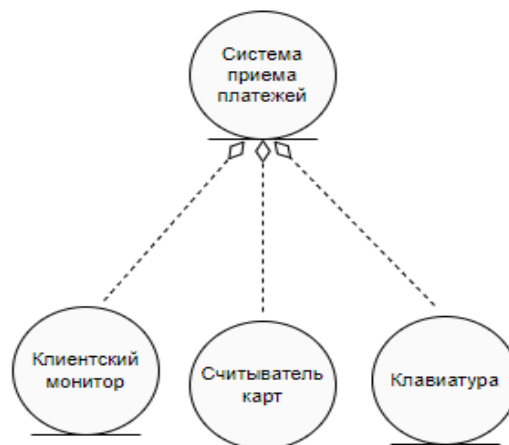


Рисунок 12. Отношение агрегации

Данное отношение, означает, что объект-целое содержит ссылку на объект-часть. Объект-часть также может содержать ссылку на объект-целое. Агрегации может указываться только между классами одного типа.

Отношение композиции аналогично агрегации, в которой «части» не могут существовать отдельно от «целого». При уничтожении объекта-«целого» должны быть уничтожены все связанные с ним объекты-«части» (рис. 13).

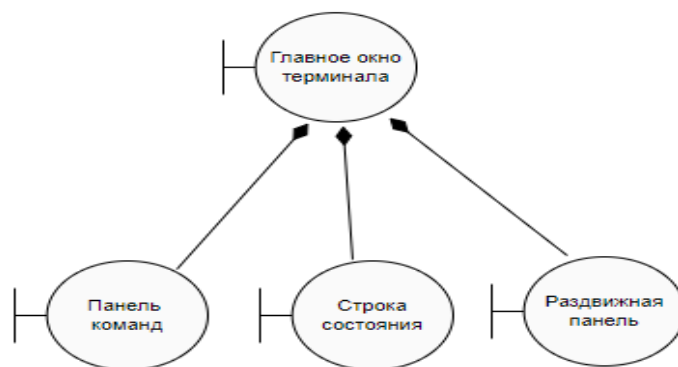


Рисунок 13. Отношение композиции

Отношение обобщения (наследования) является отношением между более общим классом и его частным случаем (рис. 14). Отношение обобщения может быть только между классами одного вида.

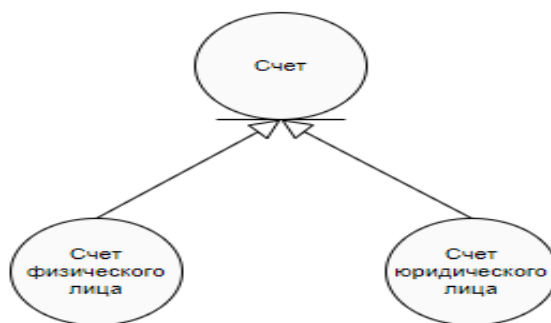


Рисунок 14. Отношение обобщения

Отношение зависимости означает, что в спецификации или теле методов объектов одного класса (зависимого) выполняется обращение к атрибутам, методам или непосредственного к объектам другого класса (независимого). Графически данное отношение обозначается штриховой стрелкой от зависимого класса к независимому классу. Данное отношение может указываться между классами анализа как одного, так и разных типов (рис. 15).

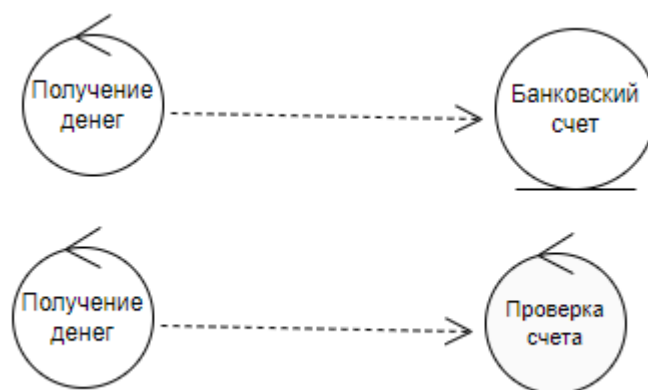


Рисунок 15. Отношение зависимости

Этапы реализации:

Шаг 1. Выполнить анализ предметной области, используя диаграмму вариантов использования. Выбрать сначала наиболее важный из перечисленных вариантов использования для включения его в модель анализа, затем по приоритету остальные.

Шаг 2. Определить основные классы анализа для выбранного варианта использования (достаточно будет исходного наброска наиболее важных для архитектуры системы). Для определения классов анализа уточните описание варианта использования в части, относящейся к внутреннему строению системы.

Шаг 3. Для каждого найденного класса определить их названия, ответственности и отношения.

Шаг 4. Разработать в программной среде модель классов анализа, установить между классами соответствующие отношения. Шаги 1-4 повторить для каждого варианта использования.

Шаг 5. Создать общую модель классов анализа, выполнить идентификацию обязанностей участвующих классов и определить отношения между ними. Выполнить исследование отношений между найденными классами, используя возможные типы связей, уделяя особое внимание классам, участвующим в разных вариантах использования и новым классам.

Шаг 6. Сохранить диаграмму, сделать выводы и оформить отчет по практической работе.

Пример построения диаграммы классов анализа для системы банкомата

В вариантах использования *работы банкомата* (рис. 16), клиент банка может, например:

- снять деньги со счета;

- перечислить деньги на другой счет;
- внести деньги на счет.

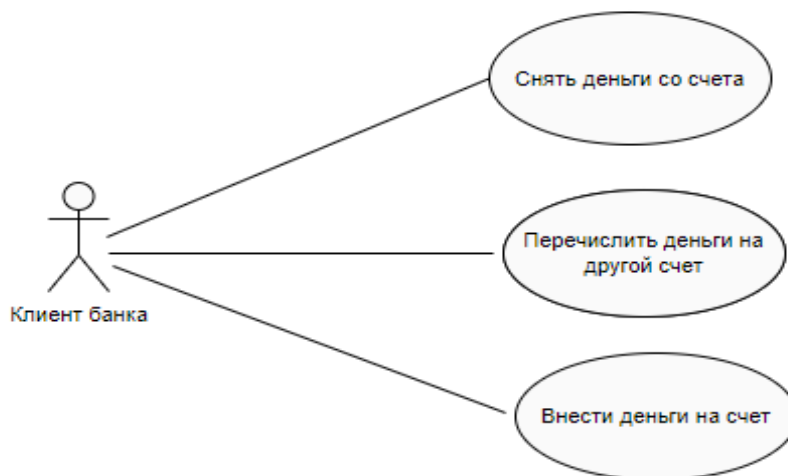


Рисунок 16. Варианты использования для работы банкомата

Шаг 1. В первую очередь выберем наиболее важный из перечисленных вариантов использования, включим его в модель анализа, определим для него классы анализа (классификаторы) и определим отношений между ними.

В качестве наиболее важного варианта использования выберем вариант «Снять деньги со счета», в котором участвуют четыре класса (рис. 17):

- устройство выдачи - граничный класс;
- интерфейс кассира - граничный класс;
- получение - управляющий класс;
- банковский счет - класс сущности.



Рисунок 17. Состав классов варианта использования «Снять деньги со счета»

Шаг 2. Установим отношения между классами варианта использования «Снять деньги со счета» (рис. 18). Для этого выберите любое из

перечисленных выше программных средств. Отрадите в рабочей области программы все определенные нами классы. При построении диаграммы старайтесь управляющий класс отображать между граничным классом и классом сущности, поскольку этот класс именно выступает в качестве посредника во взаимоотношениях между этими классами. Актер находится вне системы, а с системой всегда взаимодействует через граничный класс, то есть граничный класс является посредником между актантами (люди, другие системы) и системой, поэтому, желательно граничный класс отобразить в начале диаграммы, поскольку с него все и начинается.

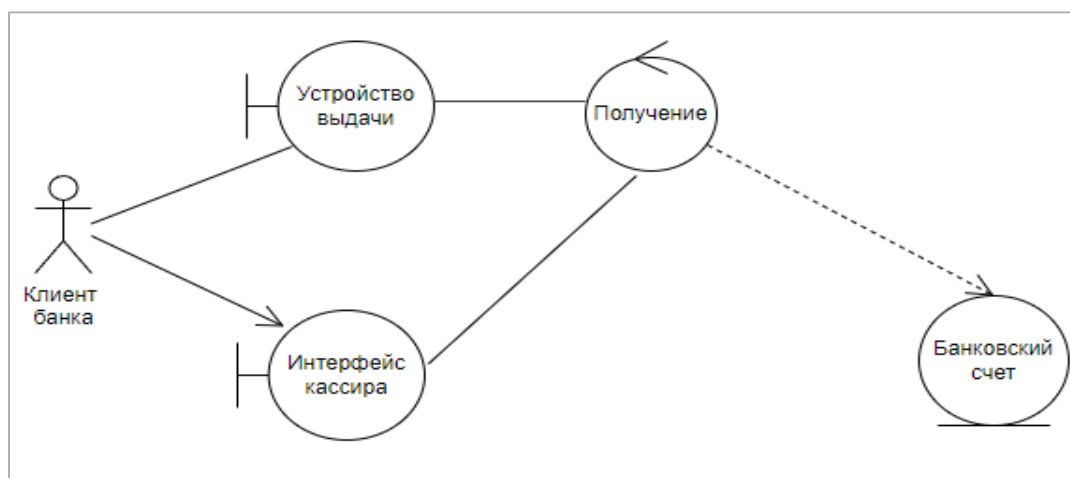


Рисунок 18. Реализации варианта использования «Снять деньги со счета» в диаграмме классов анализа

Все отношения являются ассоциацией (или направленной ассоциацией), кроме отношения между управляющим классов *Получение* и классом сущности *Банковский счет*. Управляющий класс *Получение* зависит от класса сущности *Банковский счет*, поскольку невозможно получить деньги, если на счету их нет либо не хватает.

Данный пример описывает только одну последовательность действий из возможных, а именно вариант без каких-либо осложнений. Если баланс *Банковского счета* окажется меньше запрошенной к выдаче суммы, или клиент банка не пройдет идентификацию, могут быть другие последовательности действий.

Шаг 3. Повторим первые два шага для варианта использования «Перечислить деньги на другой счет» (рис. 19), выделим классы (возможно новые классы анализа) и установим отношения между ними. В варианте использования «Перечислить деньги на другой счет» участвуют классы:

- интерфейс кассира - граничный класс;

- перечисление - управляющий класс;
- банковский счет - класс сущности.

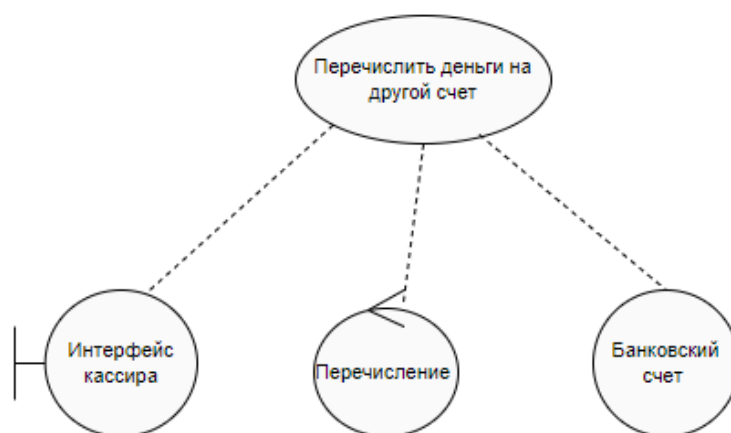


Рисунок 19. Состав классов варианта использования «Перечислить деньги на другой счет»

Установим отношения между определенными нами классами, точно также, как и в шаге 2 (рис. 20).

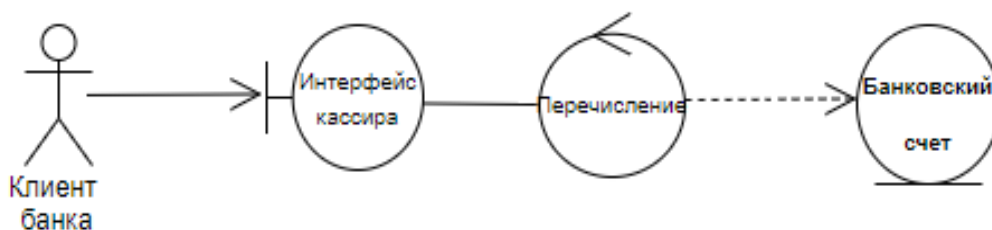


Рисунок 20. Реализации варианта использования «Перечислить деньги на другой счет» в диаграмме классов анализа

Этот пример описывает только одну последовательность из возможных действий (вариант без каких-либо осложнений) при реализации варианта использования. В другом варианте идентификация клиента могла быть неверной или, когда баланс *Банковского счета* мог оказаться меньше запрошенной к перечислению суммы.

Шаг 4. Повторяем все действия для варианта использования «Внести деньги на счет» (рис. 21). Здесь участвуют классы:

- устройство приема – граничный класс;
- интерфейс кассира - граничный класс;
- вклад – управляющий класс;

-банковский счет – класс сущности.



Рисунок 21. Состав классов варианта использования «Внести деньги на счет»

Установим отношения между классами варианта использования «Внести деньги на счет» точно также, как и в шаге 2 (рис. 22).

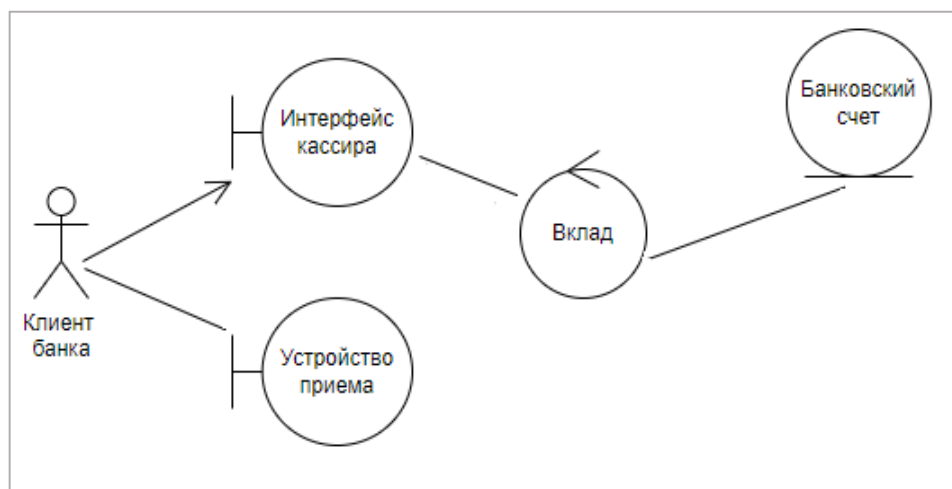


Рисунок 22. Реализации варианта использования «Внести деньги на счет»

Видим, что появились новые классы *Устройство приема* и *Вклад*.

Шаг 4. Создадим итоговую диаграмму классов анализа (рис. 23). На рисунке видим классы, участвующие в нескольких реализациях варианта использования модели анализа.

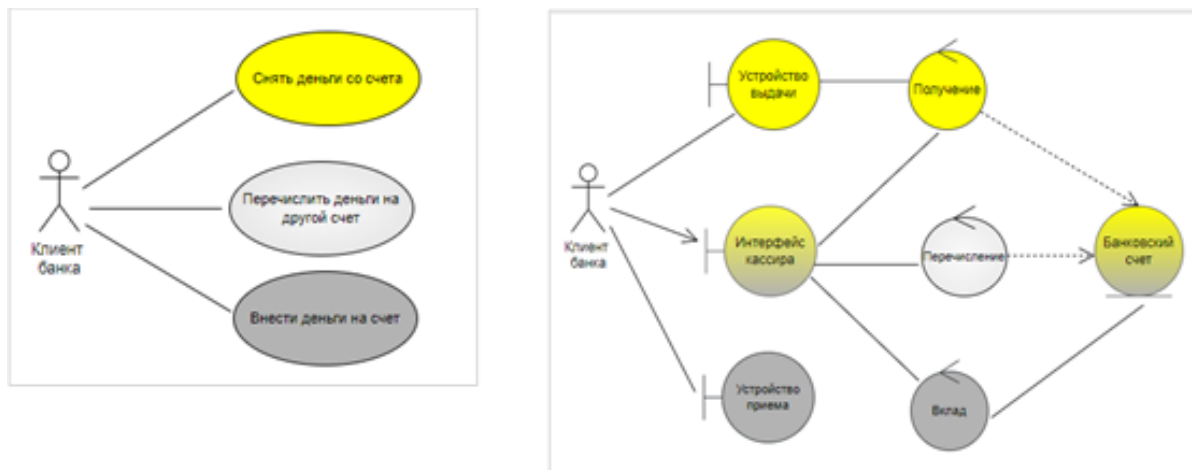


Рисунок 23. Реализация каждого из вариантов использования (слева),
в структуре классов анализа (справа)

Анализ результатов. Мы создали диаграмму классов анализа, идентифицировали обязанности участвующих классов анализа и определили отношения между ними. Интерфейс кассира, Банковский счет участвуют и исполняют роли во всех трех реализациях вариантов использования. Видим, что некоторые из классов могут оказаться новыми или изменившимися ролями уже обнаруженных классов анализа, в то время как другие роли потребуют новых классов анализа. Каждый класс анализа может участвовать и играть соответствующие роли в нескольких реализациях вариантов использования. При проектировании классов анализа по различным стереотипам требуются различные навыки и технологии. Так, например, проектирование классов сущностей обычно требует использования технологий баз данных, в то время как проектирование граничных классов — использования технологий пользовательского интерфейса. Однако классы анализа с их ответственностями, атрибутами и связями используются в качестве (логических) исходных данных для создания соответствующих операций, атрибутов и отношений классов проекта.

Правила и рекомендации построения диаграммы классов анализа

Для выделения классов сущностей необходимо определить все реальные либо воображаемые объекты, имеющие существенное значение для рассматриваемой предметной области в рамках варианта использования, информация о которых подлежит хранению.

Для каждого актера следует предусмотреть, как минимум, один граничный класс в целях организации интерфейса между ним и системой.

Для управления, обеспечения взаимодействия и координации работы объектов, реализующих одну из функций системы необходимо предусмотреть, как минимум, один управляющий класс. Как правило, взаимодействие между граничным классом и классом сущности происходит через управляющий класс.

В целях облегчения восприятия специфики связей между классами рекомендуется использовать отношения агрегации, композиции, обобщения и зависимости.

Основное внимание должно быть уделено определению (с детализацией) классов, обеспечивающих взаимодействие с внешними системами и граничным классам, обеспечивающим взаимодействие с пользователями (без детализации).

Задания для самостоятельного выполнения

Согласно индивидуальному варианту учебного проекта (п.1) постройте диаграмму классов анализа рассматриваемой системы в любой из предлагаемых программных продуктов. Для этого:

- выполните анализ предметной области, используя диаграмму вариантов использования, созданную на предыдущей практической работе;
- определите состав основных классов анализа для варианта использования, определите их названия;
- создайте общую модель классов анализа, включив структуры классов отдельных вариантов использования.

Контрольные вопросы

1. Перечислите 3 уровня абстракции классов.
2. Что собой представляет диаграмма классов аналитического уровня?
3. Что собой представляет класс анализа?
4. Для чего нужна диаграмма классов анализа?
5. Перечислите стереотипы классов анализа.
6. Как отображаются классы анализа графически?
7. Как могут быть связаны классы анализа? Перечислите названия отношений.
8. Что собой представляет отношение ассоциации?
9. Что собой представляет отношение агрегации?
10. Что собой представляет отношение композиции?
11. Что собой представляет отношение наследования (обобщения)?
12. Что собой представляет отношение зависимости?
13. Опишите пошаговый алгоритм создания диаграммы классов анализа.

3. Диаграмма последовательности

Цель работы – изучить основные элементы и правила построения, а также сформировать навык построения диаграммы последовательности в Visual Paradigm.

Задачи:

- познакомиться с назначением диаграммы последовательности, ее элементами, правилами и этапами построения;
- изучить возможности Visual Paradigm для построения диаграммы последовательности;
- построить диаграмму последовательности в Visual Paradigm в соответствии с предложенными заданиями.

Используемое ПО: Visual Paradigm, Draw.io, Rational Rose.

Краткая теория

Диаграмма последовательности описывает взаимодействие элементов какой-либо системы (например, ERP-системы) во времени, и организованы в соответствии с объектами по горизонтали и временем по вертикале. Как правило объекты размещаются слева направо в зависимости от того, в какой последовательности они вступают во взаимодействие. Время указывается в порядке последовательности и не отражает аспект длительности.

Диаграмма последовательности отражает динамический аспект взаимодействия элементов системы.

На диаграмме последовательности изображаются объекты, которые непосредственно участвуют во взаимодействии, при этом никакие статические связи с другими объектами не визуализируются. Для диаграммы последовательности ключевым моментом является именно динамика взаимодействия объектов во времени. При этом можно сказать, что диаграмма последовательности имеет два измерения. Одно - слева направо в виде вертикальных линий, каждая из которых изображает линию жизни отдельного объекта, участвующего во взаимодействии. Графически каждый объект изображается прямоугольником и располагается в верхней части своей линии жизни.

Крайним слева на диаграмме изображается объект, который является инициатором взаимодействия. Правее изображается другой объект, который непосредственно взаимодействует с первым.

Вторым измерением диаграммы последовательности является вертикальная временная ось, направленная сверху вниз. Начальному моменту времени соответствует самая верхняя часть диаграммы. Сообщения изображаются в виде горизонтальных стрелок с именем сообщения и также образуют порядок по времени своего возникновения.

Диаграммы последовательности предназначены для моделирования взаимодействия между несколькими объектами. Зачастую диаграммы последовательности создаются для моделирования взаимодействия в рамках одного прецедента.

На концептуальном уровне можно использовать диаграммы последовательности для моделирования взаимодействия между бизнес-актерами.

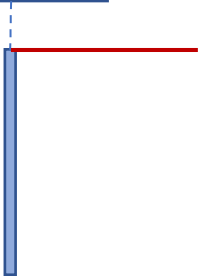
Также диаграммы последовательности подойдут для моделирования взаимодействия пользователя и информационной системы в целом.

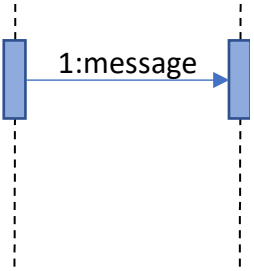
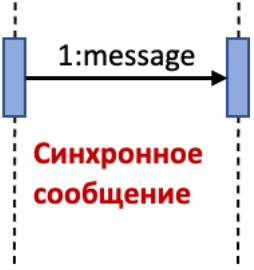
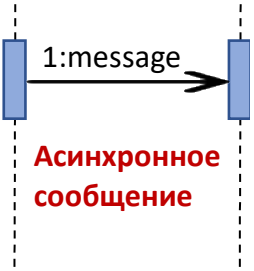
На уровне детальной спецификации требований диаграммы последовательности используются для моделирования взаимодействия компонентов системы и пользовательских классов в рамках выбранного прецедента.

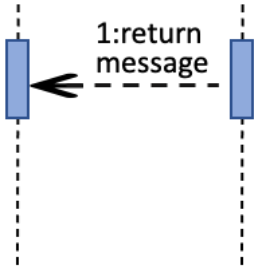
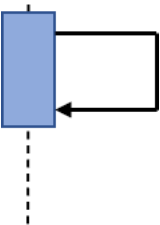
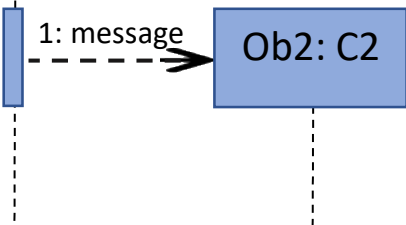
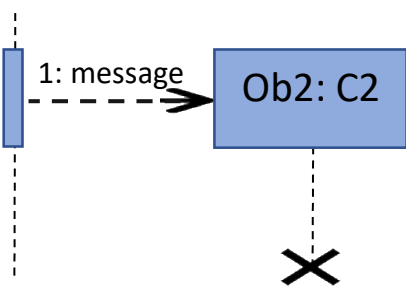
На уровне реализации с помощью диаграммы последовательности моделируется взаимодействие между отдельными компонентами информационной системы. Описание основных элементов диаграммы дано таблично (таб. 2).

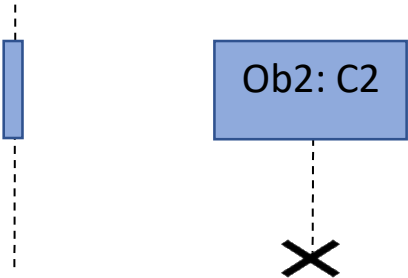
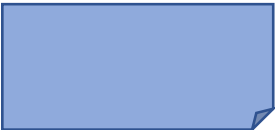
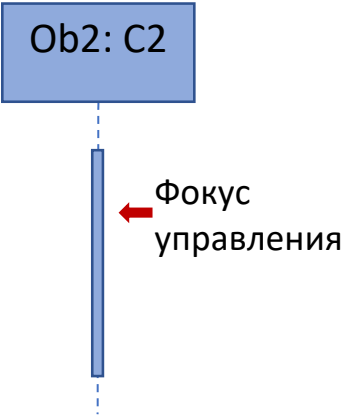
Таблица 2. Описание основных элементов диаграммы последовательности

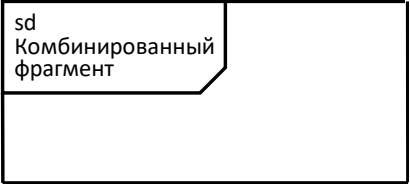
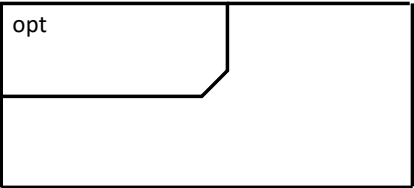
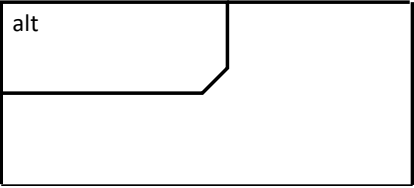
Элемент	Описание
Актер или Актор 	• Актер изображается на диаграмме последовательности самым первым объектом слева со своим фокусом управления. Актер может иметь собственное имя либо оставаться анонимным. • Тип роли, которую выполняет экземпляр, взаимодействующий с субъектом (например, путем обмена сигнала и данными). • Вне субъекта представляет роли, которые играют пользователи, внешнее оборудование или другие субъекты. • Актер не обязательно представляет конкретную физическую сущность, а просто определенную роль некоторой сущности. • Человек может играть роль нескольких разных актеров, и, наоборот, Актер может играть несколько разных ролей.

Объект Ob1: C1 Вариант 1 Ob2 Вариант 2 : C3 Вариант 3	Каждый <i>объект</i> графически изображается в форме прямоугольника и располагается в верхней части своей <i>линии жизни</i>. Внутри прямоугольника записываются собственное имя <i>объекта</i> со строчной буквы и <i>имя класса</i>, разделенные двоеточием (см. вариант 1). Объект может называться без указания класса, а также в названии объекта может быть указан только класс без имени объекта (см. варианты 2 и 3).
Линия Жизни Ob1: C1 	Это комбинация двух элементов: • прямоугольник - обозначает объект; • вертикальная линия - существование объекта в течение определенного периода времени. <i>Линия жизни</i> служит для обозначения периода времени, в течение которого <i>объект</i> существует в системе и может потенциально участвовать во всех ее взаимодействиях. Если <i>объект</i> существует в системе постоянно, то его пунктирная линия должна продолжаться по всей рабочей области <i>диаграммы последовательности</i> от самой верхней ее части до самой нижней.
Активация Ob: C1 	• Тонкий прямоугольник на линии жизни означает период, в течение которого элемент выполняет операцию. • Верх и низ прямоугольника выровнены относительно начала и завершения выполнения определенной функции.

Сообщение или вызов 	• Сообщения — это законченный фрагмент информации, который отправляется одним объектом другому. Прием сообщения инициирует выполнение определенных действий, направленных на решение отдельной задачи тем объектом, которому это сообщение отправлено. • Сообщения изображаются горизонтальными стрелками, соединяющими линии жизни или фокусы управления двух объектов. • Сообщения используются для вызова процедур, выполнения операций или обозначения отдельных вложенных потоков управления. • Начало стрелки всегда соприкасается с фокусом управления или линией жизни того объекта-клиента, который инициирует это сообщение. • Кончик стрелки соприкасается с линией жизни того объекта, который принимает это сообщение и в ответ выполняет определенные действия. При этом принимающий объект зачастую получает и фокус управления, становясь активным.
Синхронное сообщение  Синхронное сообщение	Асинхронное сообщение (с закрашенной стрелкой) используется в случаях, когда клиент посылает сообщение, например, серверу и ждет ответа перед тем, как совершить остальные действия.
Асинхронное сообщение  Асинхронное сообщение	Асинхронное сообщение (обычная стрелка), используется в случаях, когда клиент продолжает выполнять операции не дожидаясь ответа.

Ответное сообщение или возврат 	Ответное сообщение определяет конкретную связь между линиями жизни взаимодействия и представляет передачу информации вызывающей стороне соответствующего сообщения.
Найденное сообщение (found message) 	У первого сообщения нет участника, пославшего его, поскольку оно приходит из неизвестного источника. Оно называется найденным сообщением.
Потерянное сообщение (lost message)	Сообщение без адресата (есть отправитель, нет получателя).
Рекурсивный вызов 	Участник отправляет сообщение (команду) самому себе.
Создание 	В случае создания участника следует рисовать стрелку, направленную к прямоугольнику участника. Если участник будет что-выполнять сразу после создания, то сразу после прямоугольника участника нужно рисовать активацию.
Удаление 	Удаление участника обозначается большим иксом и стрелка сообщения, направленная к нему, означает, что один участник явным образом удаляет другого. Отдельные <i>объекты</i>, закончив выполнение своих операций, могут быть уничтожены, чтобы освободить занимаемые ими ресурсы. Для таких <i>объектов</i> линия жизни обрывается в момент его уничтожения. Для обозначения момента уничтожения <i>объекта</i> в языке UML применяется специальный символ в форме латинской буквы "X".

Удаление (самоудаление) 	Если же икс нарисован в конце линии жизни участника, и стрелка от другого объекта отсутствует, то это означает, что объект удаляет сам себя, другими словами, самоуничтожается.
Примечание (комментарий) 	Позволяет прикреплять различные замечания к элементам. Комментарий не несет семантической силы, но может содержать полезную информацию.
Фокус управления 	Фокус управления (<i>focus of control</i>) - специальный символ на диаграмме последовательности, указывающий период времени, в течение которого объект выполняет некоторое действие, находясь в активном состоянии. <i>Фокус управления</i> изображается в форме вытянутого узкого прямоугольника, верхняя сторона которого обозначает начало получения <i>фокуса управления объекта</i> (начало активности), а ее нижняя сторона - окончание <i>фокуса управления</i> (окончание активности). Этот <i>прямоугольник</i> располагается ниже обозначения соответствующего <i>объекта</i> и может заменять его <i>линию жизни</i>, если на всем ее протяжении он активен. Периоды активности <i>объекта</i> могут чередоваться с периодами его пассивности или ожидания. В этом случае у такого <i>объекта</i> <i>фокусы управления</i> изменяют свое изображение на <i>линию жизни</i> и наоборот. Важно понимать, что получить <i>фокус управления</i> может только <i>объект</i>, у которого в этот момент имеется <i>линия жизни</i>. Если же <i>объект</i> был уничтожен, то вновь возникнуть в системе он уже не может. Вместо него может быть создан лишь экземпляр этого же класса, который будет другим <i>объектом</i>.

Фрейм и комбинированные фрагменты 	Фрейм – это рамка, в которую вводится диаграмма. Комбинированный фрагмент (combined fragment) — элемент модели, предназначенный для представления внутренней логической структуры фрагментов взаимодействия. С помощью комбинированных фрагментов показывают различия варианты сценариев, параллельное выполнение, необязательность. UML 2.0 допускает вложения диаграмм друг в друга.
Операторы взаимодействия  	Операторы взаимодействия записываются в левом верхнем углу фрейма, и показывает различные варианты фрагмента: • opt – необязательный фрагмент (один операнд либо выполняется, либо нет); • alt – несколько альтернативных фрагментов (alternative); выполняется только тот фрагмент, условие которого истинно; • loop – это цикл, при котором фрагмент может выполняться несколько раз, а защита обозначает тело итерации; • break – завершение; • ignore (список сообщений) – игнорирование (в операнде не действует указанное сообщение); • ref – ссылка на вложенную диаграмму последовательности; • par – параллельный фрагмент; • critical – критические фрагменты внутри параллельного фрагмента; • neg – отрицание.

В дальнейшем этапы создания диаграмм последовательности рассматриваются в контексте взаимодействия пользователя, информационной системы и интернет-сайта.

Этапы реализации:

(построение диаграмм последовательности в программе Visual Paradigm)

Шаг 1. Запустить программу Visual Paradigm (рис. 24)



Рисунок 24. Запуск программы

Шаг 2. Выбрать вкладку System Design (рис. 25).

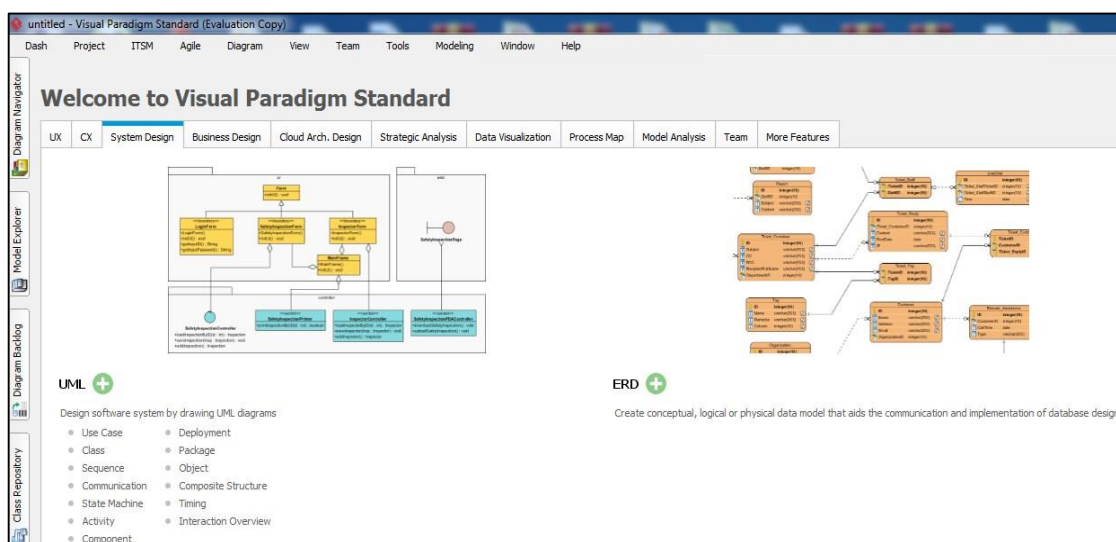


Рисунок 25. Выбор вкладки System Design

Шаг 3. Кликнуть на значок UML +

Шаг 4. Выбрать Sequence Diagram и нажать Next (рис. 26).

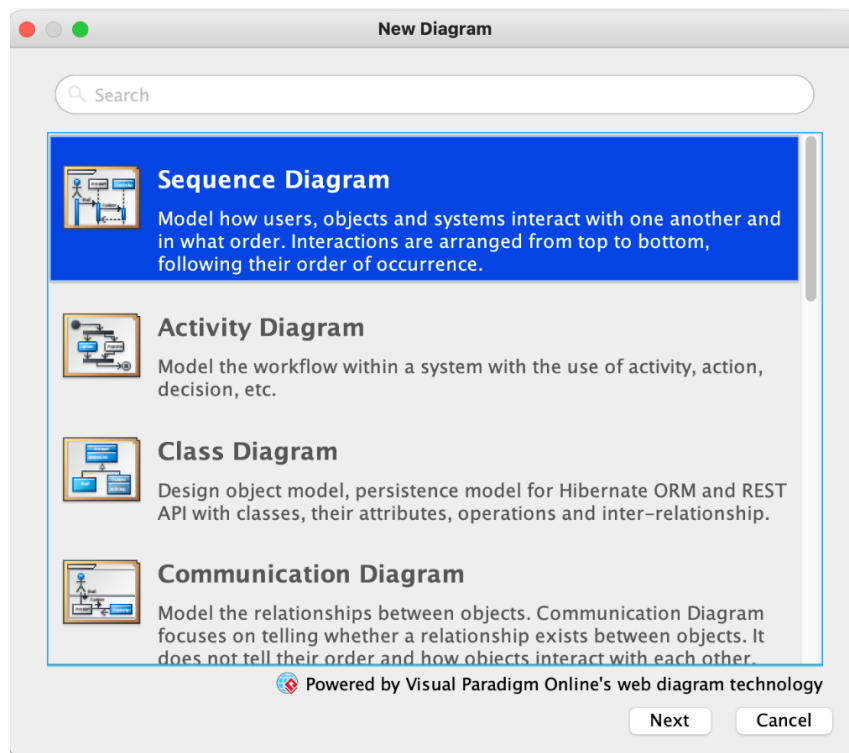


Рисунок 26. Выбор нужной диаграммы

Шаг 5. Выбрать Blank, нажать кнопку Next (рис. 27).

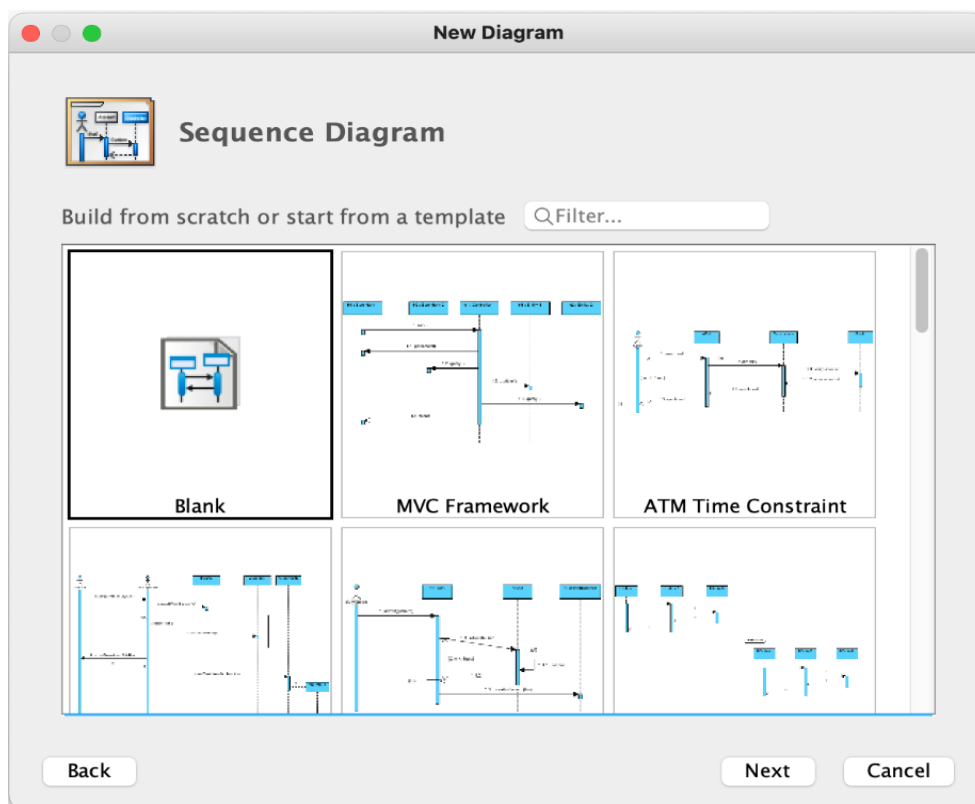


Рисунок 27. Создание новой диаграммы последовательности

Шаг 6. Задать имя диаграмме и нажать ОК (рис. 28).

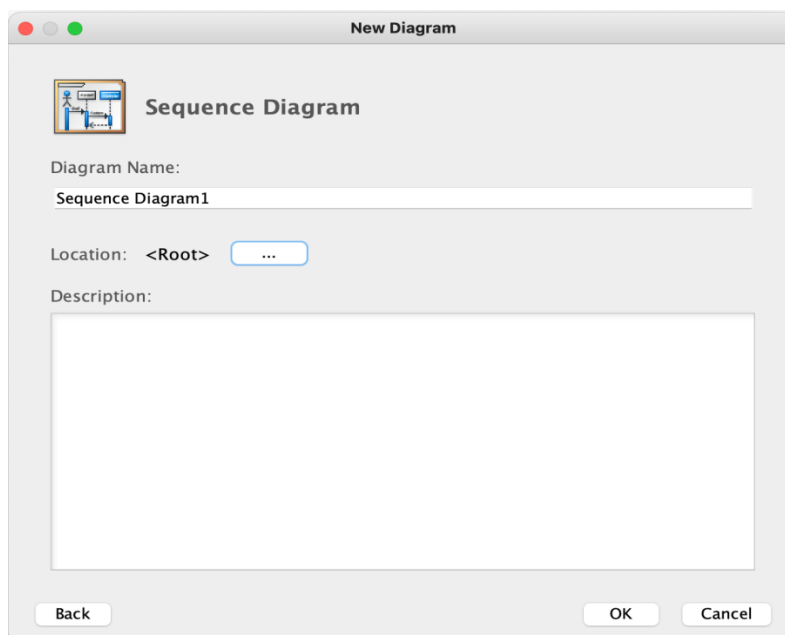


Рисунок 28. Наименование диаграммы

Шаг 7. Построить диаграмму последовательности в Visual Paradigm.

Пример построения диаграммы последовательности для описания процесса взаимодействия клиента и Интернет-магазина

В примере заказ создается через веб-сайт, а точнее через CRM-систему Интернет-магазина. Клиентом на диаграмме изображен актер по имени Покупатель. Для помещения на диаграмму Покупателя достаточно перетащить соответствующий элемент Actor (рис. 29).

Для представления CRM-системы Интернет-магазина достаточно перенести на диаграмму элемент LifeLine и переименовать ее соответствующим образом.

Для отражения запроса клиента к CRM-системе необходимо выбрать стрелку Send message из выпадающего меню с сообщениями. Затем на линии жизни CRM-системы интернет-магазина нужно разместить рекурсивное сообщения путем перетаскивания на область диаграммы элемента Recursive Message. Данное сообщение «Создать заказ» система подает сама себе. В результате создается заказ на основе запроса покупателя. Об этом система информирует клиента при помощи элемента Reply Message в виде пунктирной линии.

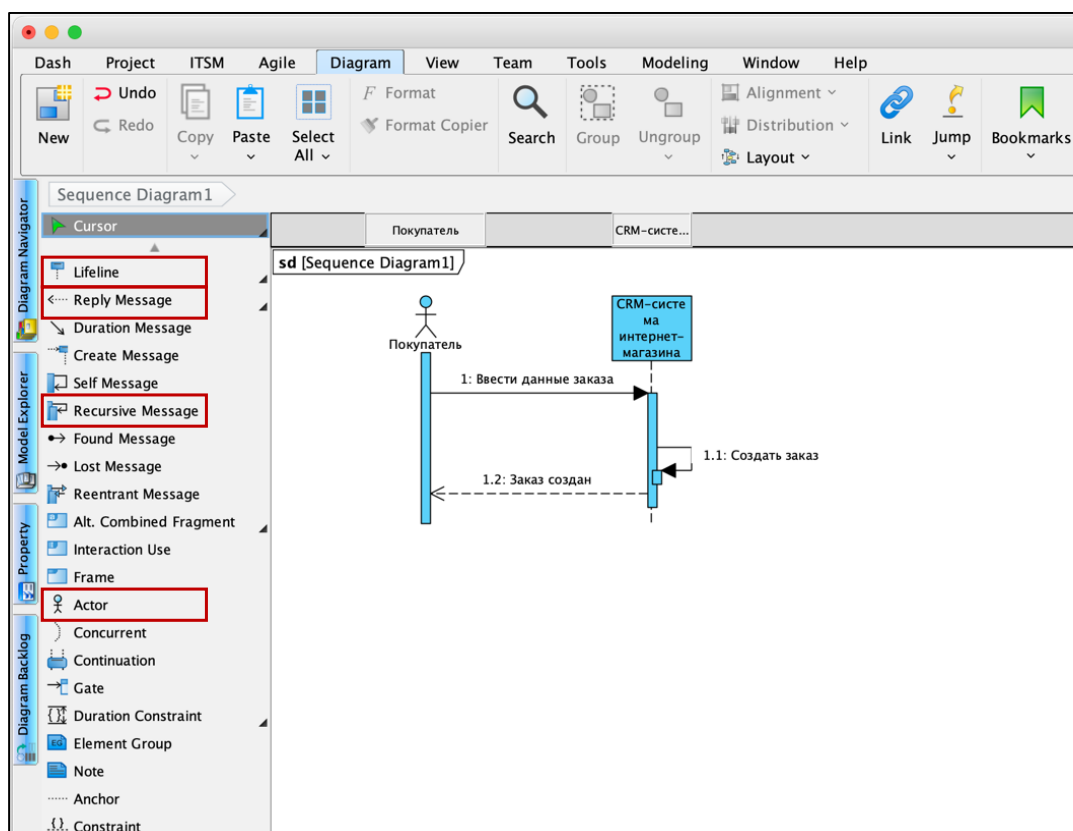


Рисунок 29. Диаграмма последовательности для описания процесса взаимодействия клиента и Интернет-магазина

На рис. 30 представлена диаграмма с оператором (или операндом) взаимодействия alt, который включает несколько альтернативных фрагментов (alternative). При этом выполняется только тот фрагмент, условие которого истинно. Оператор взаимодействия alt используется при моделировании расширенного сценария, то есть при наличии альтернативного потока взаимодействия.

Данная диаграмма описывает процесс приобретения покупателем товара в интернет-магазине с использованием кредитной карты и проверкой наличия доступного лимита денежных средств. Если лимит превышен, то покупатель будет оплачивать покупку наличными средствами курьеру в результате доставки товара.

Другими словами, диаграмма показывает реализацию одной из двух альтернатив: оплату по карте в случае наличия доступного лимита денежных средств или оплату наличными деньгами в случае превышения кредитного лимита.

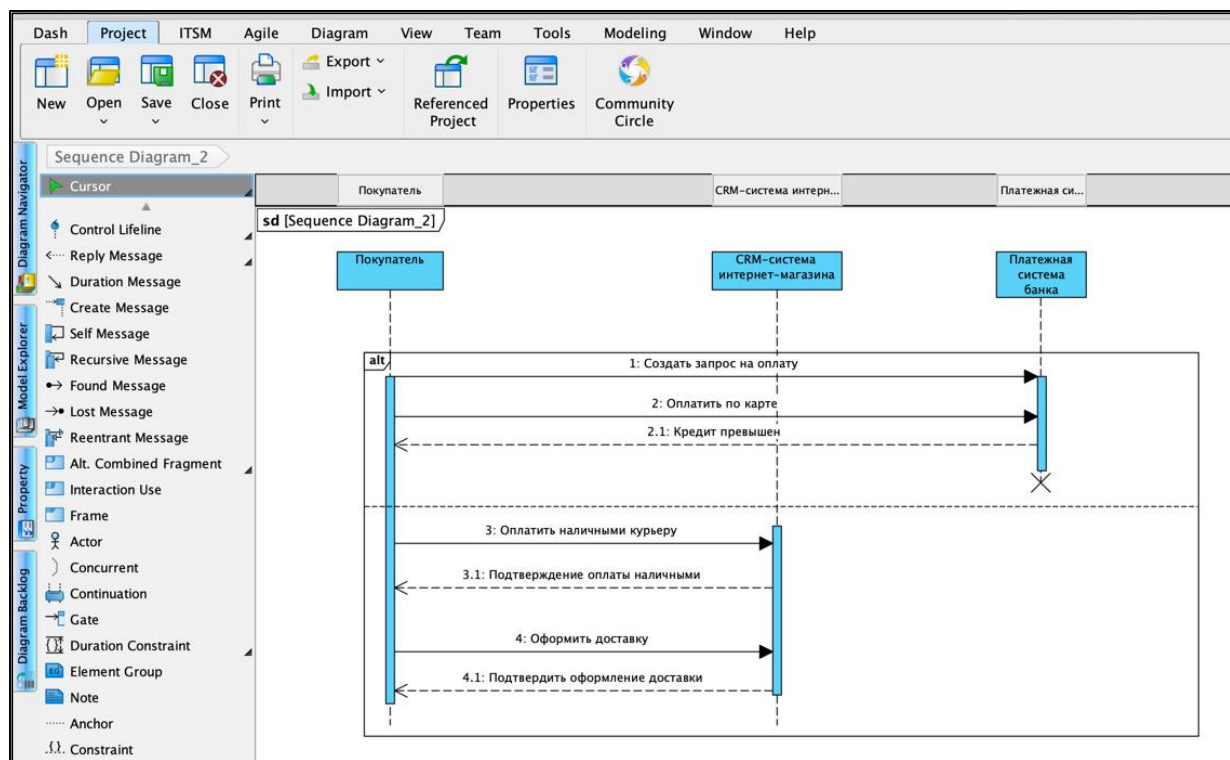


Рисунок 30. Диаграмма последовательности для описания процесса приобретения товара в интернет-магазине с использованием кредитной карты

На диаграмме с оператором взаимодействия `alt` представлено удаление объекта «Платежная система банка» при помощи специального сообщения типа `Destroy message` (см. сообщение «2. Оплата по карте»).

На рис. 31 представлена диаграмма последовательности с использованием операторов `loop`, `alt` и `opt`, которая описывает процесс обработки заказа с принятием решения об отправке заказа либо крупному, либо мелкому дистрибьютору. Данное решение принимается на основе проверки общей суммы заказа.

Как показано на диаграмме выше, в основном фреймы состоят из некоторой области диаграммы последовательности. Каждый фрейм имеет оператор. Для отображения цикла применяется оператор `loop` с единственным фрагментом. Для условной логики можно использовать оператор `alt` и помещать условие в каждый фрагмент. Будет выполнен только тот фрагмент, значение которого имеет истинное значение. Для единственной области существует оператор `opt`. Данная диаграмма последовательности начинается с сообщения типа «Найденное сообщения» отправить, что характеризует неизвестный источник отправки сообщения.

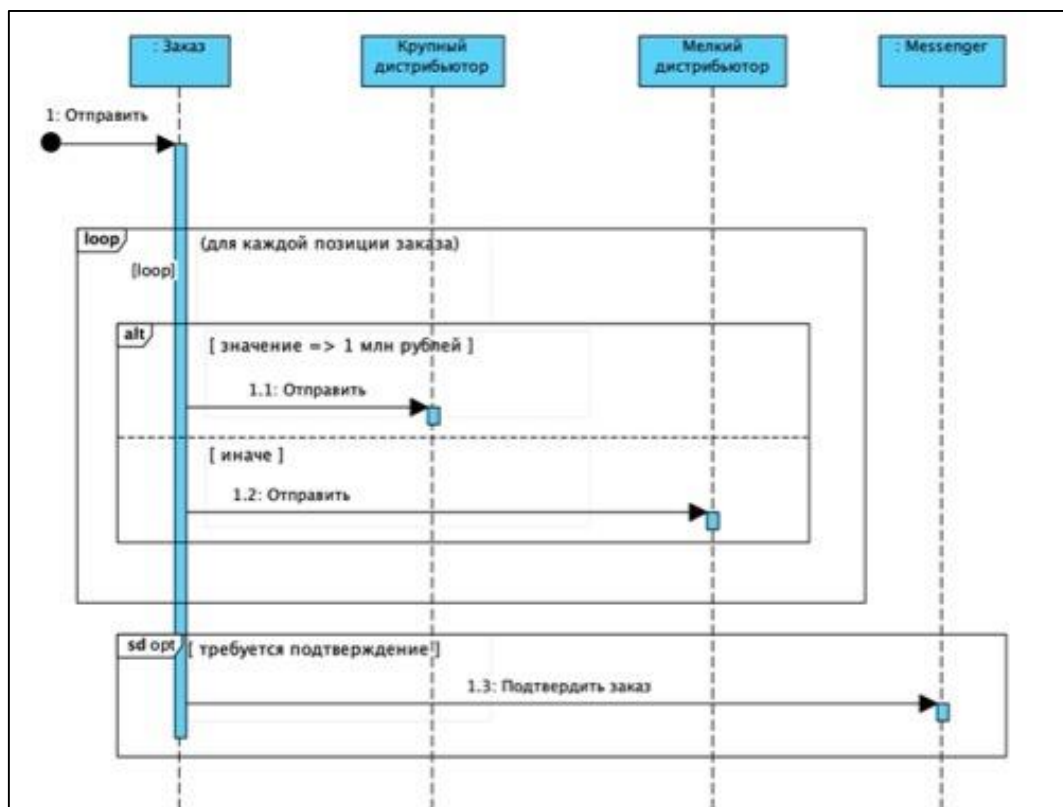


Рисунок 31. Диаграмм последовательности с использованием операторов loop, alt и opt

Ни рис. 32 представлена диаграмма последовательности для описания процесса авторизации пользователя в информационной системе с использованием оператора взаимодействия break.

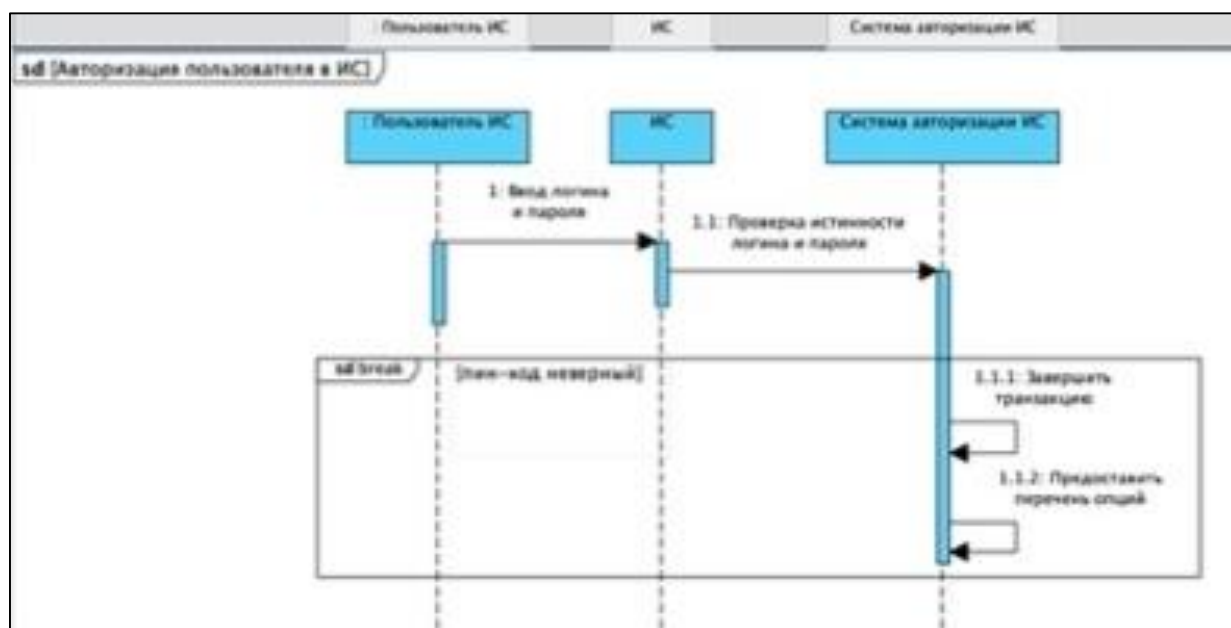


Рисунок 32. Диаграмма последовательности для описания процесса авторизации пользователя в информационной системе с использованием оператора break

Оператор взаимодействия break специфицирует комбинированный фрагмент Завершение (break), который представляет некоторый сценарий завершения.

Этот сценарий выполняется вместо оставшейся части фрагмента взаимодействия, который содержит этот соответствующий операнд.

Правила и рекомендации при построении диаграмм последовательности

Ключевые правила для построения диаграмм последовательности включают следующее:

- построение данных диаграмм целесообразно начинать с выделения из всей совокупности тех и только тех классов, объекты которых участвуют в моделируемом взаимодействии;

- все необходимые объекты должны наноситься на диаграмму с соблюдением некоторого порядка инициализации сообщений;

- необходимо установить, какие объекты будут существовать постоянно, а какие временно - только на период выполнения ими требуемых действий;

- после визуализации объектов на диаграмме следует приступить к спецификации сообщений;

- необходимо учитывать те роли, которые играют сообщения в системе. При необходимости уточнения этих ролей должны быть использованы их разновидности и стереотипы;

- для уничтожения объектов, которые создаются на время выполнения своих действий, необходимо предусмотреть явное сообщение.

Наиболее простые случаи ветвления процесса взаимодействия можно изобразить на одной диаграмме с использованием соответствующих графических операторов или операндов. Однако следует помнить, что каждый альтернативный поток управления может существенно затруднить понимание построенной модели. Поэтому общим правилом является визуализация каждого потока управления на отдельной диаграмме последовательности. В этой ситуации такие отдельные диаграммы должны рассматриваться совместно как одна модель взаимодействия.

Можно выделить шесть наиболее часто встречающихся типовых ошибок при построении диаграмм последовательностей:

Ошибка 1. Использование ненужных деталей (или объектов).

Типичная ошибка заключается в добавлении слишком большого количества деталей при работе с диаграммами последовательности. Скажем,

разработчик формирует код, который имеет довольно много ветвей (альтернативных сценариев) в определенном методе. Это не означает, что разработчик должен включать каждую из этих ветвей в одну и ту же диаграмму, используя операторы взаимодействия. Добавление слишком большого количества деталей приводит к слишком большому беспорядку, что затрудняет чтение и понимание диаграмм. То же самое можно сказать и о диаграммах последовательности на системном уровне. Главное, чтобы диаграммы были информативные и содержали только необходимую информацию, и ничего лишнего для точного описания какого-либо аспекта системы.

На рис. 33 представлен фрагмент диаграммы последовательности для описания процесса взаимодействия клиента и приложения по доставке еды. В данном случае излишне вводить в систему два объекта, которые не специфицируют клиента и приложение.

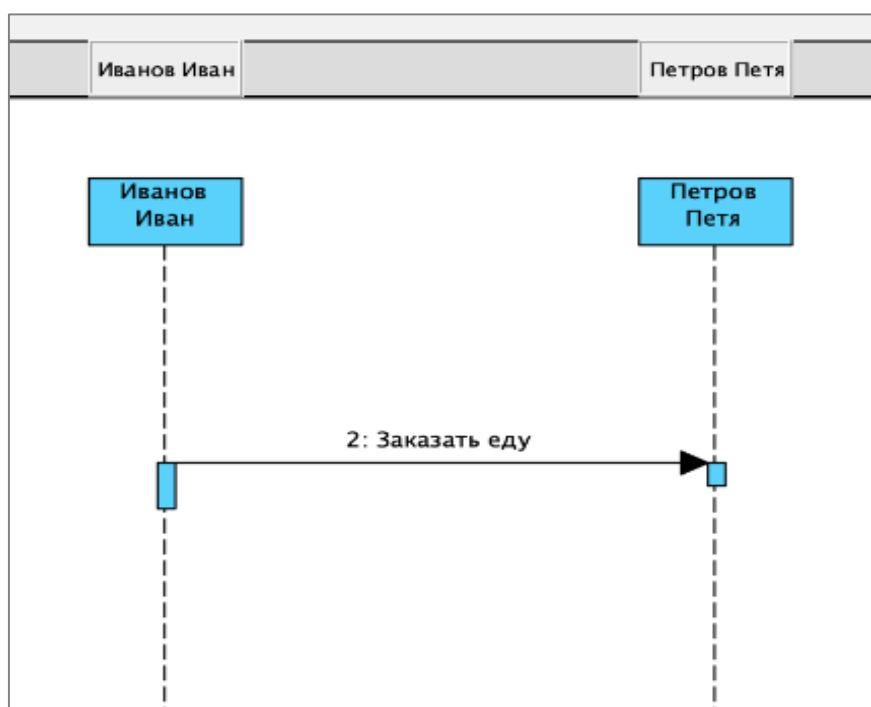


Рисунок 33. Пример ошибки использования ненужных или избыточных деталей

Ошибка 2. Нарушение принципа расположения сообщений слева направо (чаще всего).

Когда дело доходит до диаграмм последовательности, поток сообщений должен начинаться с верхнего левого угла. Поскольку в западной культуре принято читать слева направо, все классификаторы, такие как действующие лица, классы, объекты и варианты использования, должны следовать этому

пути. Однако существуют определенные исключения, когда дело доходит до этого логического потока, например, когда пары объектов вызывают операции друг над другом (см. пример данного исключения на рис. 34).

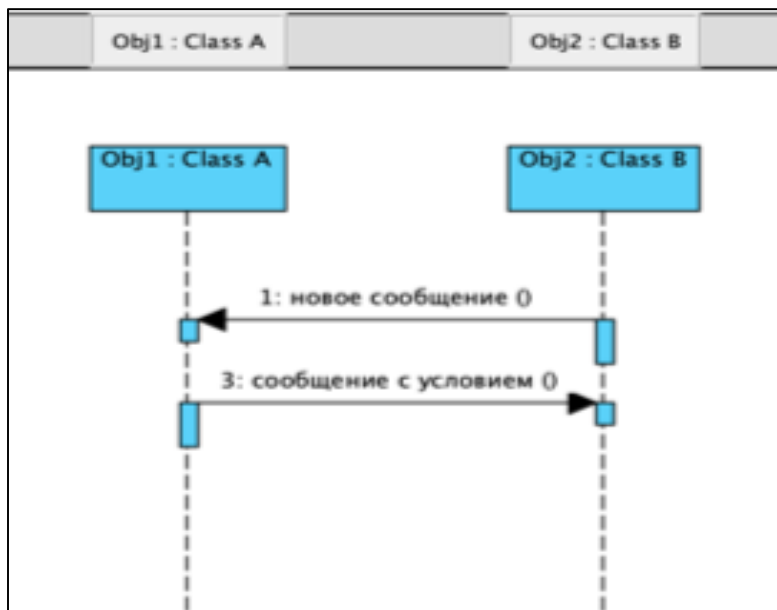


Рисунок 34. Пример обоснованного нарушения принципа расположения сообщений слева направо

Ошибка 3. Использование диаграмм последовательности, которые устарели по времени или в связи с изменениями в функционале информационной системы.

Устаревшие диаграммы последовательности, нерелевантные по сравнению с интерфейсами, фактической архитектурой или поведением системы, становятся проблемой, поскольку они перестают предлагать какую-либо ценность для документации. Это еще одна причина, по которой высокоуровневые диаграммы последовательности работают намного лучше, чем низкоуровневые. Высокоуровневые диаграммы имеют тенденцию оставаться подходящим даже при изменении деталей приложения. Им может даже потребоваться всего несколько модификаций с течением времени, чтобы оставаться актуальными.

Ошибка 4. Использование диаграмм последовательности для описания простой логики.

Одна из самых распространенных ошибок, которую совершает большинство разработчиков и аналитиков, заключается в том, что специалисты тратят драгоценное время на создание слишком большого количества диаграмм последовательности для каждого варианта использования - по одной диаграмме

для основного курса действий и по одной для каждого альтернативного варианта. Лучше всего разрабатывать диаграмму последовательности для описания достаточно сложной логики системы. Если логика проста и легко усваивается, то наличие диаграммы последовательности не принесет большой ценности по сравнению с трудозатратами на ее выполнение (рис. 35).

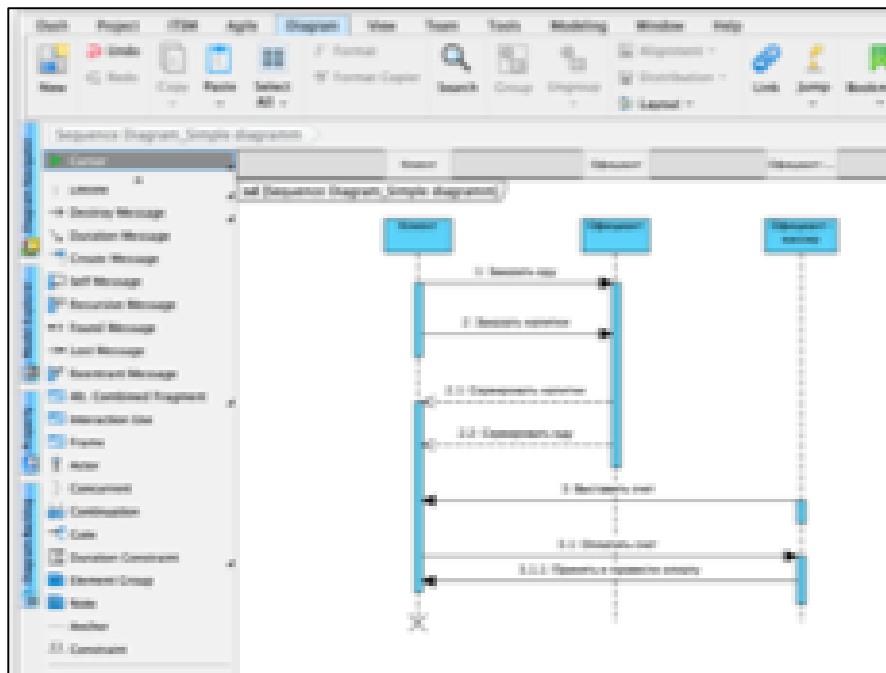


Рисунок 35. Пример использования диаграммы последовательности для описания простой логики

Ошибка 5. Обеспечьте визуальную трассировку между названиями стрелок сообщений.

Каждое предложение в тексте прецедента в идеале должно иметь вокруг себя пустое пространство. Каждое предложение также должно визуальнo гармонировать с сообщением, которое согласуется с конкретным объектом. Это позволит людям, читающим диаграмму, легко увидеть, как система будет выполнять то, что демонстрирует соответствующий объект (рис. 36).

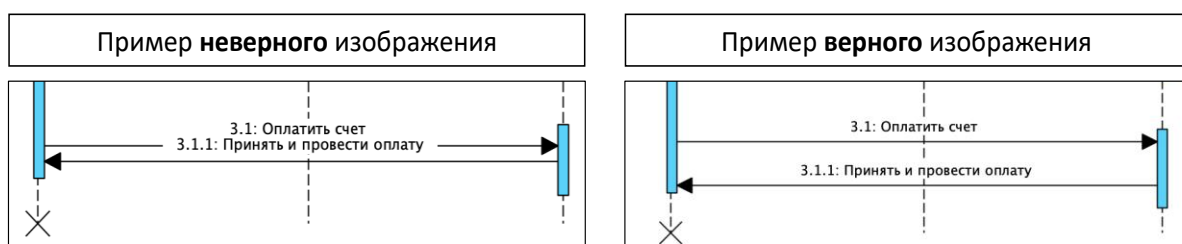


Рисунок 36. Пример верной и неверной визуальной трассировки между названиями стрелок сообщений

Ошибка 6. Принимайте во внимание распределение поведения между объектами взаимодействия системы.

При построении диаграмм последовательности одной из ключевых задач является распределение действий (или поведения) между объектами описываемой системы. По возможности действия в виде сообщений должны быть распределены по большей степени равномерно между объектами системы (рис. 37).

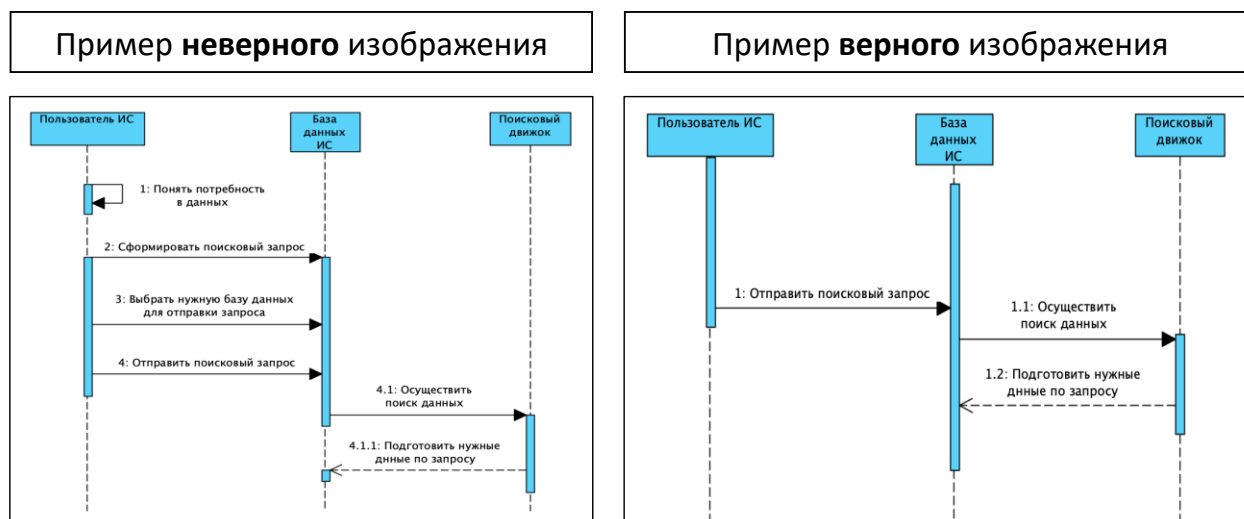


Рисунок 37. Примеры верного и неверного распределения действий (или поведения) между объектами

Понимание этих шести ошибок позволит существенно повысить качество моделирования с использованием диаграмм последовательности.

Задания для самостоятельного выполнения

1. Разработайте диаграмму последовательности для описания процесса запроса необходимых данных из базы данных информационной системы.
2. Разработайте диаграмму последовательности для описания процесса покупки товаров клиентом в Интернет-магазине.
3. Разработайте диаграмму последовательности для описания процесса снятия денег в банкомате.
4. Разработайте диаграмму последовательности для описания процесса согласования аналитиком модели бизнес-процессов с руководителем при проектировании ИС?
5. Разработайте диаграмму последовательности для описания процесса приобретения автомобиля в автомобильном салоне.

6. Разработайте диаграмму последовательности для описания процесса получения электронной книги в он-лайн библиотеке.

7. Разработайте диаграмму последовательности для описания процесса сдачи экзамена через Систему Дистанционного Обслуживания.

8. Разработайте диаграмму последовательности для описания процесса заказа такси через специальное мобильное приложение (например, Uber или Citymobil).

9. Разработайте диаграмму последовательности для описания процесса оформления проката автомобилей.

10. Разработайте диаграмму последовательности для описания процесса записи нового слушателя на курсы английского языка в онлайн-школе по изучению иностранных языков.

11. Разработайте диаграмму последовательности для описания процесса установки обновленной версии корпоративной информационной системы (например, 1С: Бухгалтерия, 1С: CRM КОРП и прочее).

12. Разработайте диаграмму последовательности для описания процесса получения потребительского кредита в банке.

13. Разработайте диаграмму последовательности для описания процесса прохождения собеседования потенциального нового сотрудника с менеджером по подбору персонала.

14. Разработайте диаграмму последовательности для описания процесса получения технической поддержки пользователя по вопросам неполадок с домашним интернетом.

15. Разработайте диаграмму последовательности для описания процесса заказа доставки готовой еды через мобильное приложения (например, ЯндексЕда и прочее).

Контрольные вопросы

1. Дайте определение понятию «диаграмма последовательности» и опишите ее назначение. Приведи примеры ситуаций, когда целесообразно ее использовать.
2. Какова роль диаграмм развертывания в проектировании информационных систем?
3. В каких случаях целесообразно использовать оператор взаимодействия alt?
4. Охарактеризуйте линию жизни объекта и представьте ее графическое изображение.
5. Для чего предназначен фокус управления? Представьте его графическое изображение.
6. Сформулировать определение понятия «сообщение». Какие виды сообщений используются в диаграмме последовательности?
7. Когда используется рефлексивное сообщение?
8. Расскажите про ветвления на диаграмме последовательности. Приведите пример ветвления.
9. В каких случаях целесообразно использовать оператор взаимодействия opt?
10. Охарактеризуйте понятие оператора взаимодействия.
11. В каких случаях целесообразно использовать операторы взаимодействия?

4. Диаграмма классов уровня проектирования (реализации)

Цель работы - научиться детально представлять внутреннюю структуру системы (программы) в виде низкоуровневых классов и связей между ними.

Задачи:

- детализировать выполненную ранее диаграмму классов анализа для классов проектирования/реализации;
- определить логическую модель представления системы с помощью различных типов связей;
- построить диаграмму и сделать вывод по представлению внутренней структуры системы.

Используемое ПО: Visual Paradigm, Draw.io, Rational Rose.

Краткая теория

С понятием диаграммы классов, хотя уже и познакомились при выполнении практической работы «Диаграмма классов анализа», здесь всё же напомним его определение, поскольку, в дальнейшем будет нами уточняться (детализироваться) диаграмму классов анализа.

Диаграмма классов (class diagram) - структурная диаграмма языка моделирования UML, демонстрирующая общую структуру иерархии классов системы, их коопераций, атрибутов, методов, интерфейсов и взаимосвязей между ними.

Как уже говорили ранее (Практическая работа «Диаграммы классов анализа»), выделяют диаграммы классов *аналитического (концептуального) уровня*, содержащие в себе только наброски общих контуров системы без привязки к среде реализации, языку программирования, *уровня проектирования (логического уровня)*, отражающие основные проектные решения касательно распределения информации и планируемой функциональности и *уровня реализации (физического уровня)*, которые дорабатывается до максимально удобного для воплощения в выбранной среде разработки вида.

Диаграммы классов анализа уже создали, научимся более детально описывать классы, то есть в данной работе будут описаны классы уровня проектирования (логического уровня).

Основными элементами диаграмм классов уровня проектирования и реализации, как и в диаграммах концептуального уровня, являются:

- классы;
- отношения.

Описание элемента «класс» относительно уровня классов проектирования и реализации

Класс - элемент диаграммы, обозначающий множество объектов, обладающих одинаковой внутренней структурой, поведением и отношениями с объектами других классов.

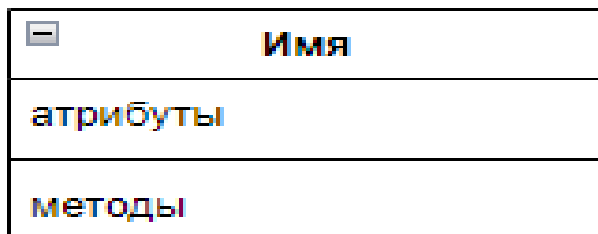


Рисунок 38. Изображение класса на диаграмме

Изображается класс на диаграмме в виде прямоугольника, разделённого на три секции, то есть в рамках, содержащих три компонента – имя, атрибут операция/метод. По сути изображение класса выглядит как горизонтально вытянутый прямоугольник (с одним, двумя, тремя разделами/секциями). Пример класса с тремя секциями изображен на рис. 38.

Верхняя часть диаграммы, в которой написано имя класса называется *секцией имени*. Секция имени имеет следующие характеристики:

- в качестве имени класса выбирается существительное в единственном числе;
- если имя класса состоит из нескольких слов, то оно записывается в верблюжьем стиле (от англ. "CamelCase");
- имя класса выравнивается по центру и пишется полужирным шрифтом;
- имя класса начинается с заглавной буквы;
- если класс абстрактный — то его имя пишется полужирным курсивом.

Посередине располагаются поля (атрибуты) класса – это есть *секция атрибутов*. Секция атрибутов имеет следующие характеристики:

- атрибуты выровнены по левому краю;
- атрибуты начинаются с маленькой буквы.

Нижняя часть класса содержит операции (методы) класса – это есть *секция операций/методов*. Секция операций/методов имеет следующие характеристики:

- операции/методы выровнены по левому краю;
- операции/методы пишутся с маленькой буквы.

Секция «имя»

Имя класса должно быть уникальным в рамках пакета. *Имя является обязательным элементом* обозначения класса на диаграмме.

Абстрактный класс – это класс, на основе которого нельзя создать объекты. Такие классы используются в качестве шаблона для дочерних классов при наследовании. Имя абстрактного класса пишется курсивом.

Статический класс - это класс, в котором есть только статические поля и методы и на основе которого не создаются объекты. Чтобы показать на диаграмме, что наш класс статический, нужно добавить к имени модификатор «utility». Это значит, что объекты этого класса не создаются.

В секции имени класса может быть указан стереотип (например, «entity», «boundary», «interface», «control», «utility» и т. п.).

Секция «атрибуты»

Атрибут — это свойство класса, которое может принимать множество значений. Множество допустимых значений атрибута образует домен. Атрибут имеет имя и отражает некоторое свойство моделируемой сущности, общее для всех объектов данного класса. Класс может иметь произвольное количество атрибутов. На рисунке 39 изображен класс с двумя атрибутами.

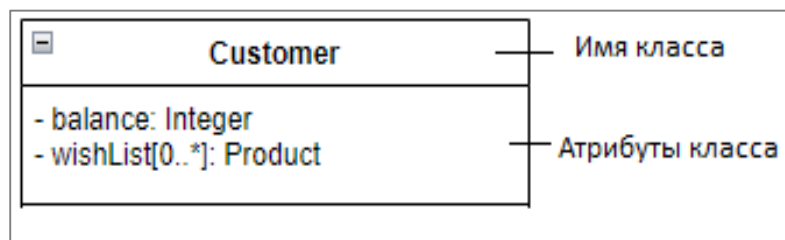


Рисунок 39. Класс с атрибутами

Атрибуты могут содержать огромный набор спецификаций (набор параметров, которыми характеризуются сами атрибуты) в отдельной строке:

[видимость] [/] имя [: тип [['кратность']] [= исходное значение]]
[{'модификаторы'}].

Для атрибута *обязательным является наличие имени*, все остальные спецификации являются дополнительными, то есть вариативными.

Видимость (англ. visibility) указывает на то, что атрибут может быть прочитан или изменен с помощью объектов других классов. Причем, изменить можно только в том случае, если атрибут константой не является.

Видимость может быть отображена как с помощью символов, так и с помощью обозначений:

- "+"/Public – указывает на общедоступность атрибута как для чтения, так и для модификации атрибутами других классов;

- "#"/Protected – указывает на защищенность атрибута, когда он доступен только объектам текущего класса, а также его потомкам;

- "-"/Private – указывает на закрытость атрибута, когда доступность атрибута обеспечивается объектам текущего класса;

- "~"/Package – указывает на доступность атрибут только объектам классов текущего пакета.

Производность (символ «/») перед именем атрибута указывает на то, что он является производным.

Имя атрибута является текстовой строкой, которая его идентифицирует. Имя должно быть уникальным внутри класса. Но в разных классах имена атрибуты могут быть и одинаковыми. Например, атрибут с именем ФИО в классе «преподаватель» подразумевает ФИО преподавателя, а в классе «студент» ФИО студента. Но одинаковые имена атрибутов внутри одного класса не могут быть!!

Тип атрибута выбирается исходя из семантики значений, которые должны храниться в атрибуте, и, как правило, возможностей целевого языка программирования по представлению этих значений.

Кратность атрибута характеризует количество значений, которые можно хранить в атрибуте.

Модификатор описывает особенности реализации атрибута.

Секция «операции»/«методы»

Операция/методы — реализация функции, которую можно запросить у любого объекта класса. Операция показывает, что можно сделать с объектом. Исполнение операции часто связано с обработкой и изменением значений атрибутов объекта, а также изменением состояния объекта. На рис. 40 изображен класс с двумя атрибутами и с четырьмя операциями/методами.

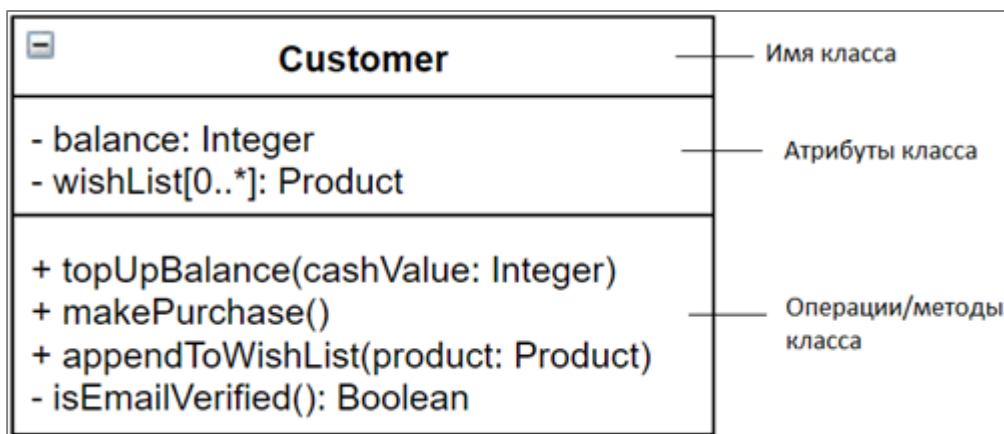


Рисунок 40. Класс с атрибутами и методами

Существуют следующие виды операций (методов):

- *конструктор*;
- *деструктор*;
- *модификатор*;
- *селектор*;
- *итератор*;
- *событие*.

Описание элемента «отношение» относительно классов уровня проектирования и реализации

Подробно отношения между классами описывали в практической работе «Диаграммы классов анализа». Связи классов уровня проектирования и реализации осуществляются по тем же правилам, что и в диаграммах классов концептуального уровня, поэтому повторять определения, описывать визуальное отображение и подробные характеристики отношений в этой работе не будем (см. теорию к практической работе «Диаграммы классов анализа»).

Напомним только, что между классами имеется шесть видов отношений:

- *ассоциация*;
- *агрегация*;
- *композиция*;
- *наследование*;
- *реализация*;
- *зависимость*.

Этапы реализации:

Шаг 1. *Выбрать свой вариант задания из представленных в учебных вариантах. За основу взять диаграмму классов анализа, созданную на основе*

вариантов использования, а также, если есть возможность, использовать созданные на предыдущих практических работах другие диаграммы. Это позволит точнее описать классы.

Шаг 2. *Наметить основные классы*, детально спроектировав укрупненные абстракции классов анализа и уточнив описание варианта использования в части, относящейся к внутреннему строению системы.

Шаг 3. *Изобразить классы на диаграмме*, задав пока только имена классов.

Шаг 4. *Определить и изобразить виды классов* (абстрактные, статические и др.) диаграмме.

Шаг 5. *Определить (описать) атрибуты* классов.

Шаг 6. *Добавить операции/методы* класса.

Шаг 7. *Отобразить связи* между классами (можно указать кратность).

Шаг 8. *Создать итоговую диаграмму* и сохранить ее.

Пример создания детальной диаграммы классов для программы построения графиков функций

Основной функционал программы может быть отражен на ранее созданной диаграмме вариантов использования (рисунок 3), а также ранее созданной диаграмме классов анализа (буду использовать диаграмму вариантов использования).

Функционал программы показывает, что пользователь может:

- определить константу с заданием его имени и тел;
- определить функцию с заданием тела, значений параметров и ограничений на значение аргумента;
- построить график функции.

Шаг 1. За основу построения диаграммы классов возьмем варианты использования.

Шаг 2. Наметим основные классы, для этого выделим основные механизмы, на основе которых можно построить классы:

- механизм, ответственный за работу с математическими функциями будут предоставлять возможность создавать объекты математических функций и подсчитывать значения.
- механизм, ответственный за список токенов, будут разбивать строковое представление выражения на список токенов.
- механизм, позволяющий строковое представление математических выражений разбить на элементарные части, называемые токенами будут создать постфиксную форму выражения из списка токенов

- механизм для определения и использования именованных констант, например, $\pi = 3,14$ или $e = 2,71$ будет очень доставлять удобство использования готовых констант для пользователей.
- механизмы обработки ошибок позволят проверять пользовательские выражения на корректность математической записи.
- механизм построения графика функции на координатной плоскости.

Для этих механизмов можно определить следующие классы:

- класс для хранения математического выражения - *MathExpression*.
- класс для разбиения строкового представления выражения на список токенов - *MathParser*.
- класс для построения постфиксной формы выражения из списка токенов - *MathFormConverter*.
- класс для работы с именованными константами - *MathConstantManager*.
- класс для проверки корректности пользовательского математического выражения - *MathChecker*.
- класс для подсчёта значений математического выражения - *MathCalculator*.
- класс для построения графика функции - *Graph*.

Желательно на диаграммах уровня реализации описывать классы на английском языке, чтобы не запутаться при сопоставлении написанных названий классов с элементами на диаграмме. Дело в том, что одно и то же слово разные программисты могут перевести по-разному. Например, класс "МатематическоеВыражение" некоторые могут перевести как "MathExpression", другие, исходя из специфики задачи, как "Function" или просто как "Expression".

Шаг 3. Теперь изобразим эти классы на диаграмме, задав пока только имена классов (рис. 41).

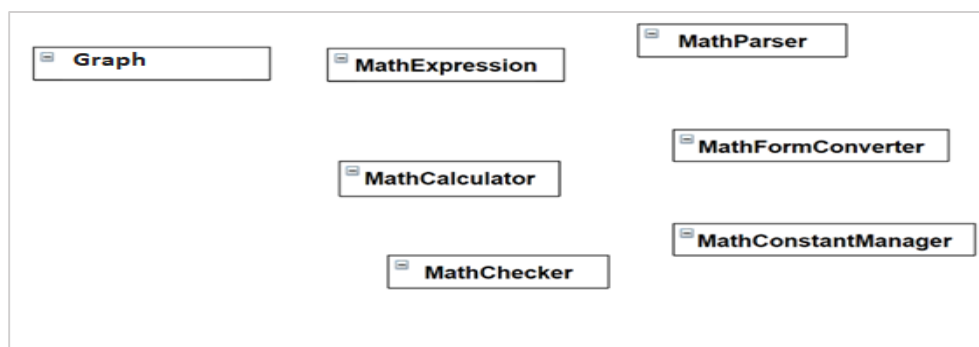


Рисунок 41. Имена классов

Шаг 4. Определим виды классов (абстрактные, статические и др.) и изобразим их на диаграмме (рис. 42).

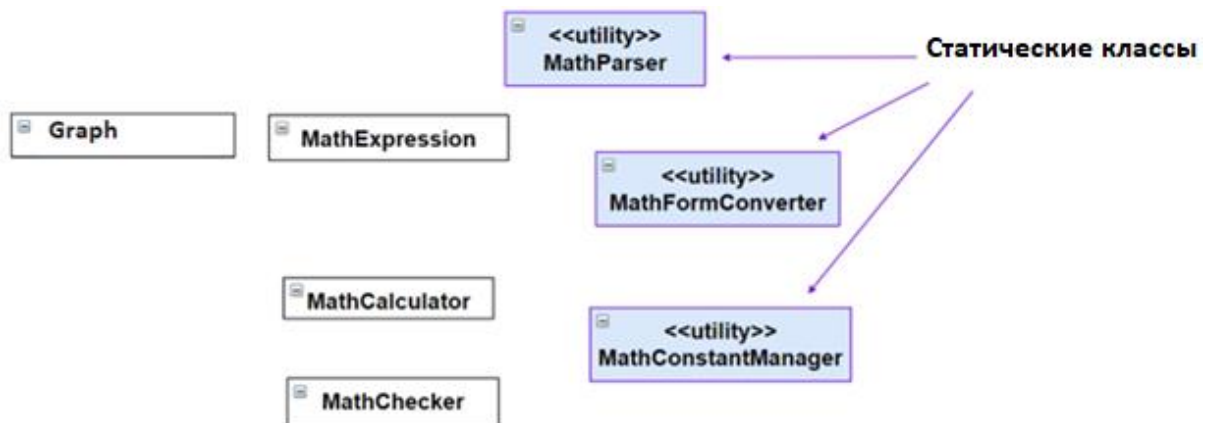


Рисунок 42. Выделение и обозначение статических классов

Классы *MathParser*, *MathFormConverter*, *MathConstantManager* являются статическими, потому что они представляют собой «сборник» полезных функций, которые мы объединили в класс. Изобразим это на нашей диаграмме.

Шаг 5. Определим атрибуты для классов (рис. 43).

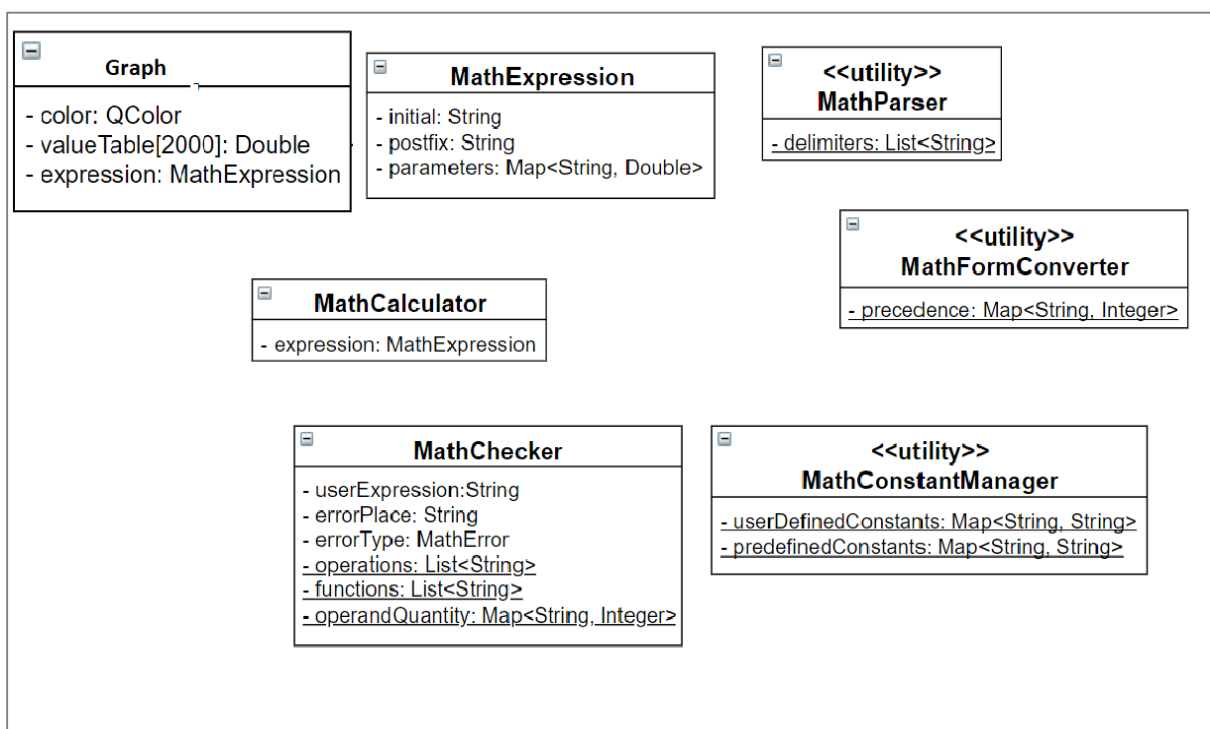


Рисунок 43. Классы с атрибутами

Шаг 6. Добавим операции/методы классов (рис. 44).

Graph
- color: QColor - valueTable[2000]: Double - expression: MathExpression
+ calculateTable()

MathExpression
- initial: String - postfix: String - parameters: Map<String, Double>
+ setParameter(parameter: Char, value: Int) + setExpression(expression: String) + getStringRepresentation(): String

<<utility>> MathParser
- delimiters: List<String>
+ CreateTokenList(expression: String): List<String>

<<utility>> MathFormConverter
- precedence: Map<String, Integer>
+ InfixToPostfix(infixExpression: String): String

MathCalculator
- expression: MathExpression
<<constructor>> + MathCalculator(expression: MathExpression) + calculate(variableValue: Double): Double

<<utility>> MathConstantManager
- userDefinedConstants: Map<String, String> - predefinedConstants: Map<String, String>
+ AddConstant(constant: String, value: String) + AlterConstantValue(constant: String, newValue: String) + DeleteConstant(constant: String) + GetConstantValue(constant: String): String + IsConstant(token: String): Boolean

MathChecker
- userExpression: String - errorPlace: String - errorType: MathError - operations: List<String> - functions: List<String> - operandQuantity: Map<String, Integer>
<<constructor>> + MathChecker(expression: String) + areAllTokensCorrect(): Boolean + areBracketsCorrespond(): Boolean + hasEmptyBrackets(): Boolean + hasMissedOperations(): Boolean ... + IsOperation(token: String): Boolean ...

Рисунок 44. Классы с атрибутами и операциями

Шаг 7. Отобразим связи между классами (рис. 45). Для этого рассмотрим различные отношения между классами на диаграмме. В первую очередь добавим отношения ассоциации, затем остальные.

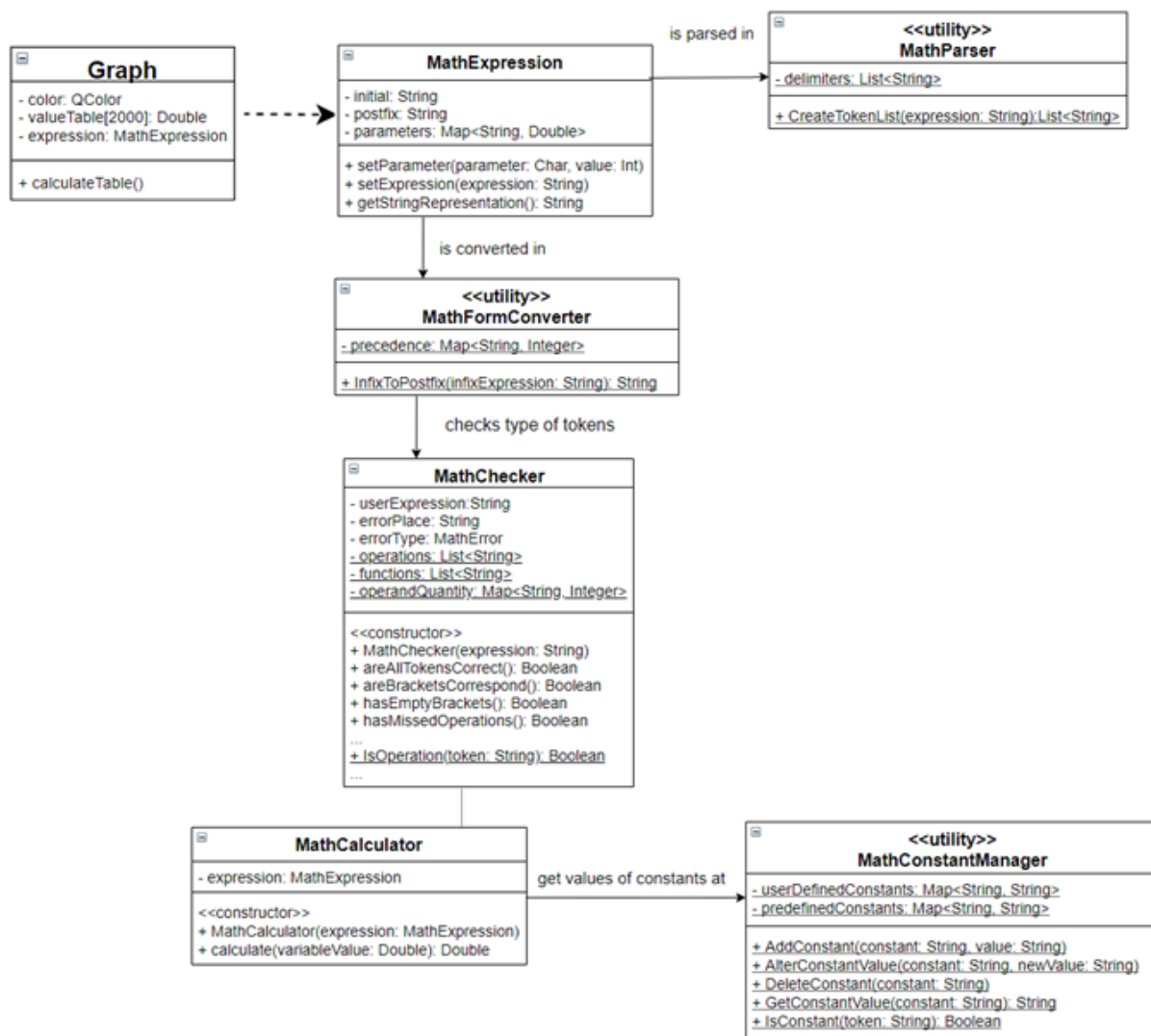


Рисунок 45. Диаграмма классов с указанием связей

Основные виды связей здесь определяются как ассоциативные. Только вид графика функции зависит объекта математического выражения. Объекты класса *Graph* зависят от изменений в объектах класса *MathExpression*, поскольку, в объекте *Graph* хранится указатель на объект класса *MathExpression*. Все классы нашей диаграммы, кроме класса *Graph* «работают» на объекты класса *MathExpression*. Людям, работающим с графической частью приложения, нужно знать лишь об этом классе, что существенно снижает сложность.

Шаг 8. Создадим итоговую диаграмму с указанием кратностей (рис. 46).

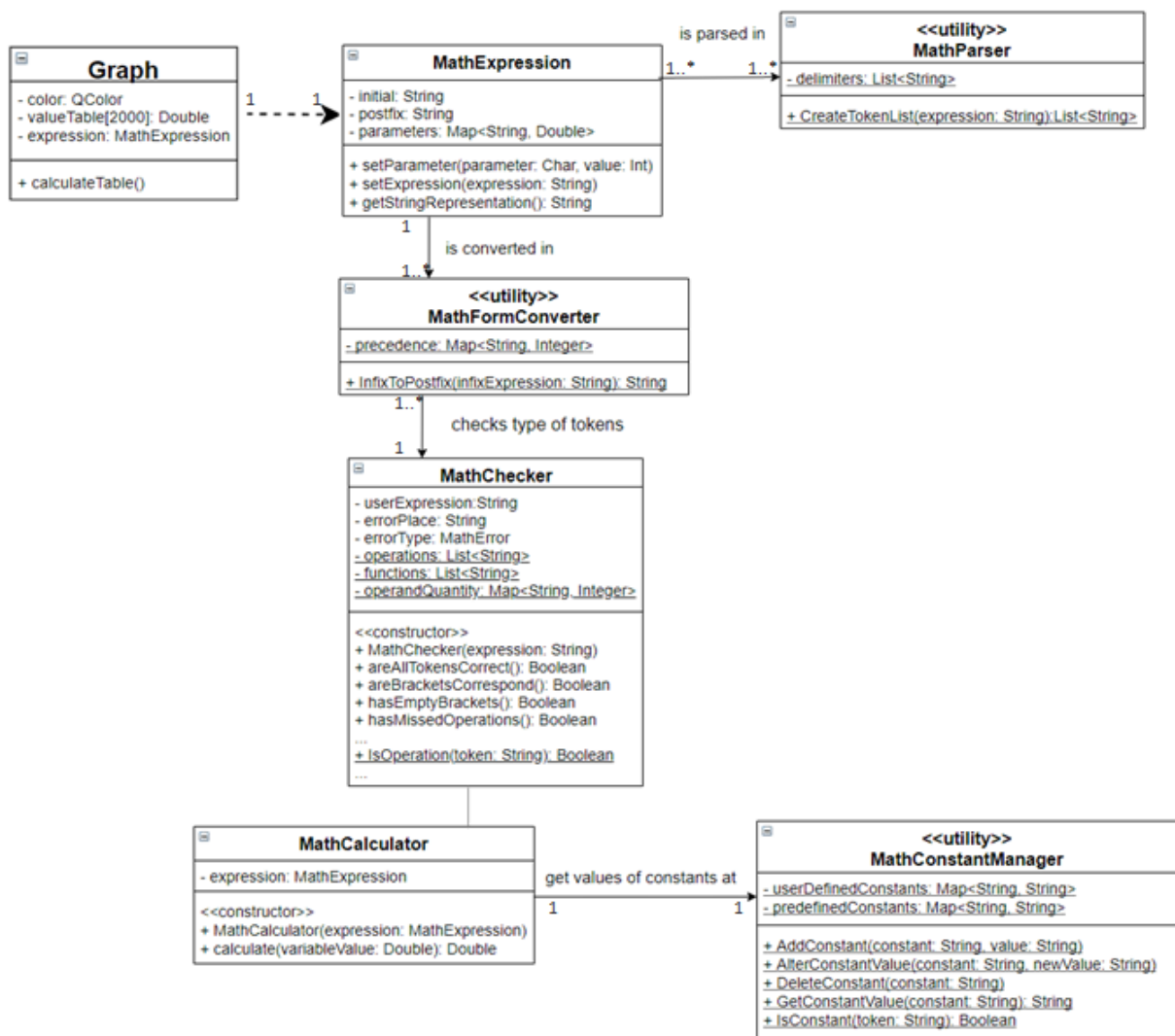


Рисунок 46. Итоговая диаграмма с указанием кратности

Анализ результатов. Мы создали диаграмму классов детального уровня. Для этого определили сем классов для построения диаграммы: *MathExpression*, *MathCalculator*, *MathParser*, *MathFormConverter*, *MathConstantManager*, *MathChecker*, *Graph*. Выделили статические классы, добавили атрибуты и описали методы, установили отношения между классами, указали кратности. Основные отношения между классами – ассоциация. Класс *Graph* и *MathExpression* связаны между собой отношением зависимости объектов класса *Graph* от изменений в объектах класса *MathExpression*.

Создадим таблицы (табл. 3, табл. 4) для наглядного анализа диаграммы.

Таблица 3. Описание классов диаграммы

Название класса	Описание
<i>MathExpression</i>	Класс для хранения математического выражения
<i>MathParser.</i>	Класс для разбиения строкового представления выражения на список токенов
<i>MathFormConverter</i>	Класс для построения постфиксной формы выражения из списка токенов
<i>MathConstantManager</i>	Класс для работы с именованными константами
<i>MathChecker</i>	Класс для проверки корректности пользовательского математического выражения
<i>MathCalculator.</i>	Класс для подсчёта значений математического выражения
<i>Graph</i>	Класс для построения графика функции

Таблица 4. Взаимодействие между классами

Класс	Кратность	Тип отношения	Класс
<i>MathExpression</i>	Многие к многим	ассоциация	<i>MathParser</i>
<i>MathExpression</i>	Один к многим	ассоциация	<i>MathFormConverter</i>
<i>MathFormConverter</i>	Многие к многим	ассоциация	<i>MathChecker</i>
<i>MathChecker</i>	Один к одному	ассоциация	<i>MathCalculator.</i>
<i>MathCalculator</i>	Один к одному	ассоциация	<i>MathConstantManager</i>
<i>MathConstantManager</i>	Один к одному	ассоциация	<i>MathCalculator</i>
<i>Graph</i>	Один к одному	зависимость	<i>MathExpression</i>

Правила и рекомендации построения диаграммы классов анализа

1. За основу диаграммы детальных классов рекомендуется взять диаграмму классов анализа, если она создана, или вариантов использования, если нет диаграммы классов анализа, а также диаграммы взаимодействия (последовательности и коммуникации).

2. Если классов в системе много, рекомендуется распределить их по пакетам и для каждого пакета разработать отдельную диаграмму классов, а в общей диаграмме указать связь между пакетами.

3. Для классов должны быть определены и специфицированы все атрибуты и методы. Их спецификация, как правило, выполняется с учетом выбранного языка программирования.

4. При определении методов рекомендуется использовать сообщения с ранее разработанных диаграмм последовательности и коммуникации.

5. Граничные классы, определяющие интерфейс взаимодействия с пользователем (пользовательский интерфейс) как правило, не требуют детализации. Однако, граничные классы, определяющие интерфейс взаимодействия с другими системами, требуют детального проектирования.

6. Основное внимание следует уделять классам «сущности» (особенностям отображения информации). Можно применять подходы, используемые при проектировании БД.

7. Несмотря на то, что каждому объекту при выполнении программы автоматически назначается уникальный идентификатор, рекомендуется для классов-сущностей явно определять атрибуты, хранящие значения первичного ключа.

8. Поощряется использование в классах многозначных атрибутов в виде массивов, множеств, списков и т. д.

9. Следует уделять внимание также и управляющим классам. Однако, надо помнить, что управляющие классы надо проектировать только в случаях крайней необходимости. Лучше этот класс поделить между граничными классами и классами-сущностями.

Задания для самостоятельного выполнения

Согласно индивидуальному варианту учебного проекта постройте диаграмму классов рассматриваемой системы в любой из предлагаемых программных продуктов. Для этого используйте пошаговый алгоритм разработки диаграммы классов, описанный выше. Варианты заданий выбираются в 1 практической работе.

Контрольные вопросы

1. Чем отличается класс уровня проектирования от класса уровня реализации?
2. Опишите характеристики секции имени класса.
3. Что собой представляет атрибут класса?
4. Опишите характеристики секции атрибута класса.
5. Чем отличается абстрактный класс от статического класса?
6. Опишите набор спецификаций атрибута.
7. Что собой представляет видимость атрибута?
8. Что собой представляет тип атрибута?

9. Что собой представляет кратность (множественность)?
10. Перечислите варианты указания кратности.
11. Что собой представляет модификатор?
12. Что собой представляет операция/методы класса?
13. Опишите характеристики секции методов класса.
14. Опишите набор спецификаций методов.
15. Что собой представляет тип параметра?
16. Что собой представляет тип метода?
17. Чем отличается тип параметра от типа метода?
18. О чем могут сказать свойства метода?
19. Перечислите основные виды операций.
20. Опишите алгоритм создания диаграмм классов уровня проектирования/реализации.

5. Диаграмма деятельности

Цель работы – сформировать навык построения диаграммы деятельности в Visual Paradigm.

Задачи:

- ознакомиться с назначением диаграммы деятельности и ее элементами;
- изучить возможности Visual Paradigm для построения диаграммы деятельности;
- построить диаграмму деятельности в Visual Paradigm в соответствии с предложенными заданиями.

Используемое ПО: Visual Paradigm, Draw.io, Rational Rose.

Краткая теория

При разработке модели поведения проектируемой системы необходимо не только представить процесс изменения ее состояний, но и отразить особенности реализации выполняемых системой операций. Для моделирования процесса выполнения операций в языке UML используются диаграммы деятельности (активности).

Диаграммы деятельности (activity diagram) служат для моделирования последовательности действий, которые выполняются различными элементами, входящими в состав системы.

Диаграммы деятельности, или активности, предназначены для уточнения вариантов использования и моделей последовательности. Диаграммы активности показывают поток управления, подобно диаграммам последовательности, но сосредоточивают внимание на операциях, а не на объектах. На рис. 47 представлен пример диаграммы деятельности.

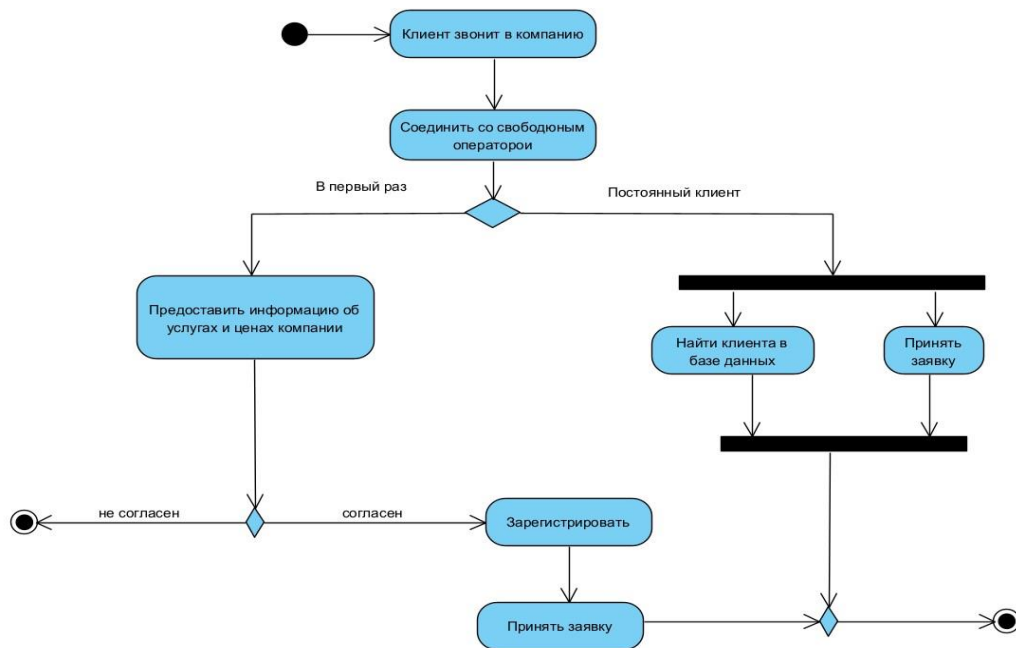
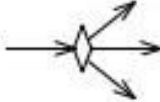
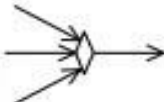
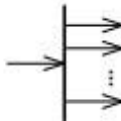
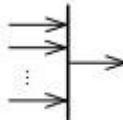
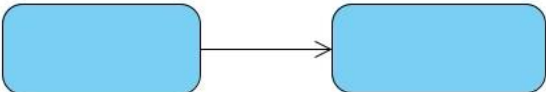


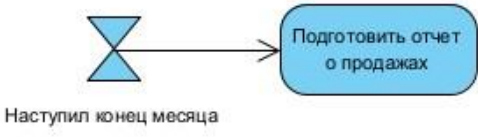
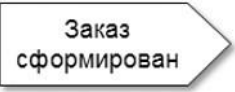
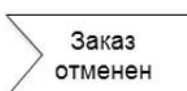
Рисунок 47. Пример диаграммы деятельности

Графические узлы и пути на диаграмме деятельности представлены в табл. 5.

Таблица 5. Графические узлы и пути на диаграмме деятельности в Visual Paradigm

Тип графического элемента	Нотация
Деятельность (activity)	
Деятельность (activity) - пример	
Действие (action)	
Действие (action) - пример	

Начальный узел (initial node)	
Узел финала деятельности	
Узел решения (decision node)	
Узел слияния (merge node)	
Узел разделения (fork node)	
Узел соединения (join node)	
Поток управления (control flow)	
Поток объектов (object flow)	
Заметка, пояснение (note)	
Разбиение деятельности (activity partition)	

Событие времени / Действие приема временного события (accept time even action)	
Событие времени / Действие приема временного события (accept time even action) - пример	
Действие передачи сигнала (send signal action)	
Действие передачи сигнала (send signal action) – пример	
Действие приема события (accept even action)	
Действие приема события (accept even action) - пример	

Этапы реализации:

Шаг 1. Запустить программу Visual Paradigm (рис. 48)

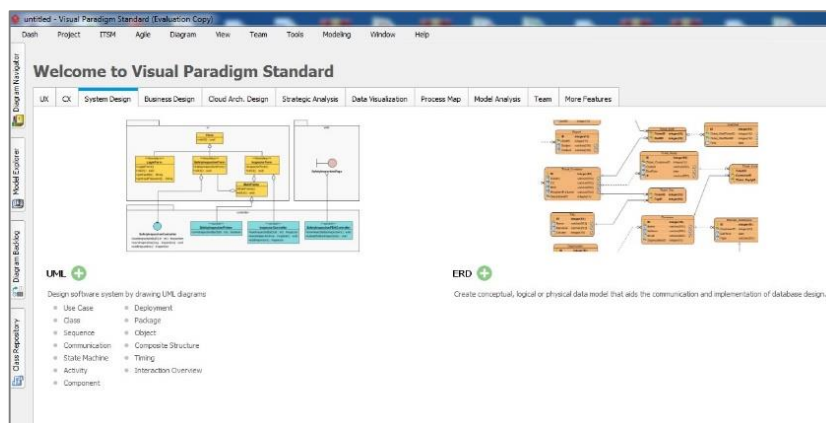


Рисунок 48. Главная форма

Шаг 2. Выбрать вкладку System Design (рис. 49)

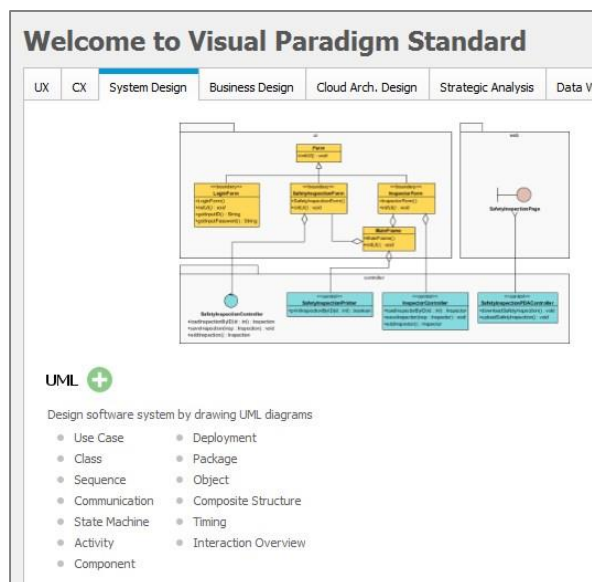


Рисунок 49. Выбор вкладки

Шаг 3. Выбрать Activity Diagram и нажать Next (рис. 50).

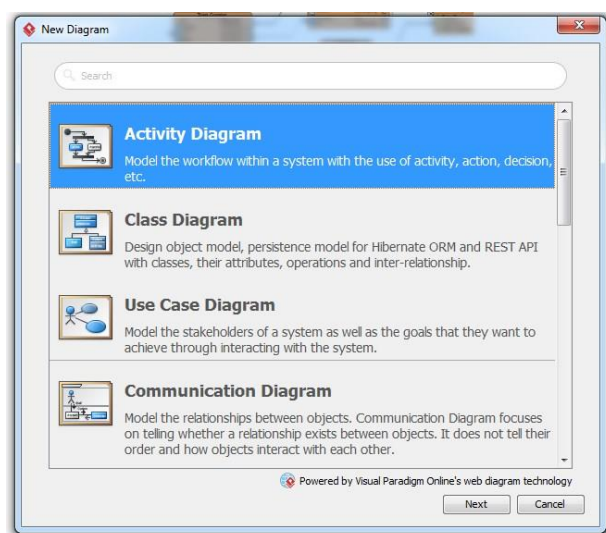


Рисунок 50. Выбор типа диаграммы

Шаг 4. Выбрать шаблон «Plannig Show», нажать Next.

Шаг 5. Получить для работы шаблон соответствующей формы.

Шаг 6. Модернизировать шаблон.

Примеры диаграмм деятельности

При модернизации шаблона следует добавить на диаграмму узел решения, узел слияния, изменить узел разделения и узел соединения. Для получения стрелок, идущих от элемента действия, к элементу решения (узел слияния), следует использовать стрелку типа Control Flow (CF)  Control Flow. Для получения стрелок, изогнутых под углом 90 градусов, следует довести стрелку до места

изгиба, кликнуть левой кнопкой мышки и далее довести стрелку до нужного элемента диаграммы. Диаграммы деятельности могут быть использованы для моделирования бизнес-процессов. Применительно к бизнес-процессам желательно выполнение каждого действия ассоциировать с конкретным подразделением компании. В этом случае подразделение несет ответственность за реализацию отдельных действий, а сам бизнес-процесс представляется в виде переходов действий из одного подразделения к другому (рис. 51).

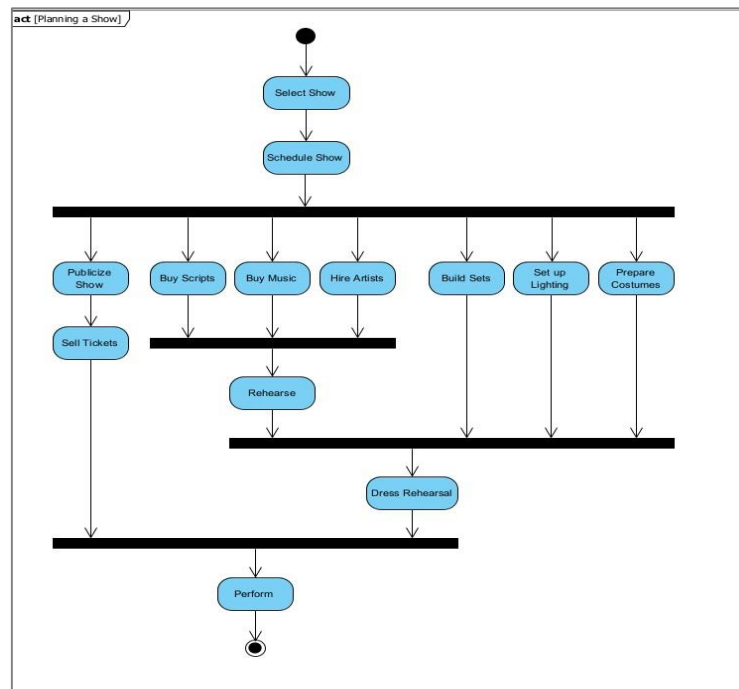


Рисунок 51. Пример диаграммы деятельности

Для моделирования этих особенностей в языке UML используется специальная конструкция, получившая название дорожки (swimlanes). При этом все состояния действия на диаграмме деятельности делятся на отдельные группы, которые отделяются друг от друга вертикальными линиями. Две соседние линии и образуют дорожку, а группа состояний между этими линиями выполняется отдельным подразделением.

Названия подразделений явно указываются в верхней части дорожки. Пересекать линию дорожки могут только переходы, которые в этом случае обозначают выход или вход потока управления в соответствующее подразделение компании.

На рис. 52 представлен пример диаграммы деятельности с дорожками.

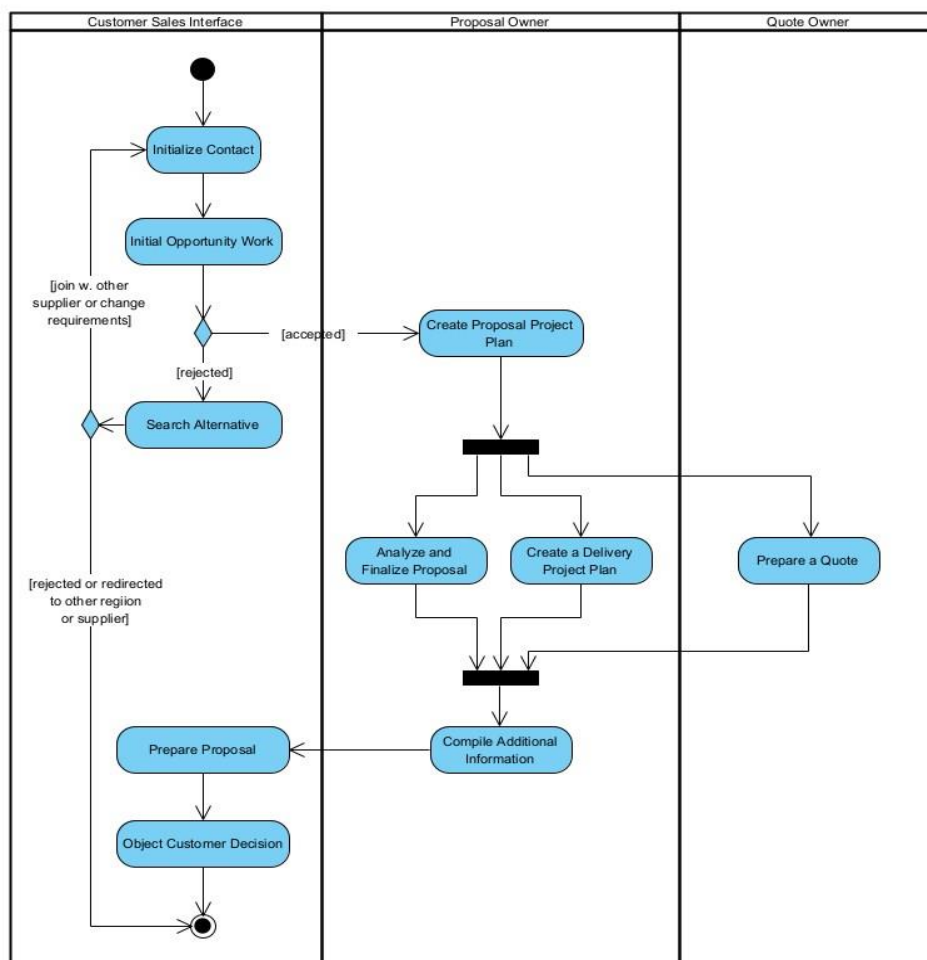


Рисунок 52. Пример диаграммы деятельности с дорожками

Правила и рекомендации при построении диаграммы деятельности

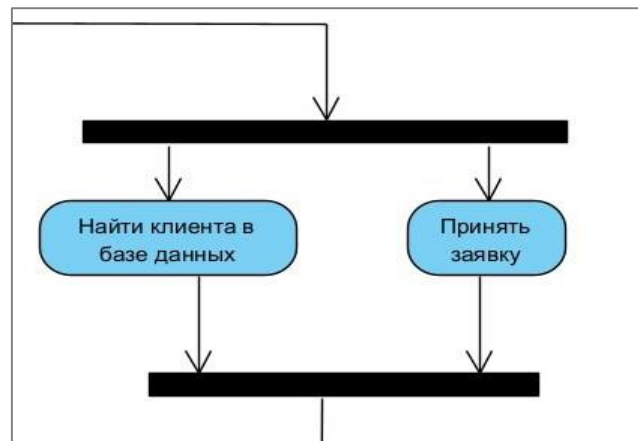
1. При построении диаграмм рекомендуется использовать классические принципы моделирования – декомпозиции и иерархического упорядочения. При моделировании алгоритма вначале строится контекстная диаграмма с деятельностью, которые после детализируются с помощью соответствующих диаграмм декомпозиции.

2. Количество пересечений линий следует минимизировать. При этом считается, что пересекающиеся линии не имеют логической связи друг с другом. Другими словами, потоки данных или управления в местах пересечений не меняют своего направления.

3. Если на диаграмме имеется ветвление/решение на параллельные или альтернативные потоки, то должно указываться и соответствующее соединение/слияние этих потоков.

4. В целях определения зоны ответственности (набора действий) сущностей рекомендуется использовать разделы деятельности.

5. В одно действие может входить только один поток управления (рис. 53). Ошибкой является вариант построения диаграммы деятельности, при котором в одно действие входят два или более потоков управления, либо вариант построения диаграммы деятельности, при котором потоки управления соединяются не через слияния.



*Рисунок 53. Правильное ветвление
потока управления*

Задания для самостоятельного выполнения

1. Разработайте диаграмму деятельности для системы контроля покупок в книжном магазине. Система обеспечивает сканирование каждой книги и вычисляет общую сумму покупки. Если штрих-код на книге поврежден, то кассир вводит стоимость книги вручную. Система позволяет клиенту расплатиться наличными или по кредитной карте. После подтверждения оплаты система контроля покупок печатает чек и сохраняет данные о сделанных покупках. Если клиент хочет вернуть ранее купленный товар в магазин, то предъявляет чек, подтверждающий сделанную покупку. Кассир принимает товары, вводит в систему отмену покупки по предъявленному чеку и выдает наличные клиенту. При этом система ведет учет возвращенных покупок.

2. Для описанной ниже задачи разработайте диаграмму деятельности. Распределите ответственность при помощи плавательных дорожек. Покажите взаимодействия между подразделениями. Пользователь принимает решение модернизировать свой компьютер и купить DVD-проигрыватель. Он начинает со звонка в отдел продаж производителя компьютера, откуда его направляют в службу поддержки. Служба поддержки сообщает пользователю о моделях и возможных вариантах установки DVD-проигрывателя. Пользователь выбирает модель проигрывателя и заказывает доставку почтой. Он получает

проигрыватель, успешно устанавливает его в компьютер, после чего отправляет по почте оплаченный счет.

3. Разработайте диаграмму деятельности приготовления кофе с использованием кофе-машины.

4. Разработайте диаграмму деятельности для алгоритма нахождения корней квадратного уравнения.

5. Разработайте диаграмму деятельности для оформления заказа в Интернет-магазине с использованием двух дорожек: заказчик, интернет-магазин. Предусмотреть три варианта оплаты: наличными при получении, оплата картой при получении, онлайн-оплата на сайте. Предусмотреть два варианта доставки: в пункте выдачи заказов и доставка курьером.

6. Разработайте диаграмму деятельности для перевода с карты на карту с использованием банковского приложения

7. Разработайте диаграмму деятельности выдачи кредитной карты клиенту банка – физическому лицу.

8. Разработайте диаграмму деятельности выдачи ипотечного кредита клиенту банка – физическому лицу.

9. Разработайте диаграмму деятельности, описывающую в подробностях процесс входа в систему электронной почты. При разработке диаграммы следует обратить внимание на то, что ввод пароля и имени пользователя может осуществляться в произвольном порядке.

10. Для описанной ниже задачи разработайте диаграмму деятельности. Покажите зоны ответственности при помощи плавательных дорожек. Покажите изменения состояния проекта по мере выполнения деятельности. Компания производит новый продукт и должна координировать несколько отделов. Продукт начинает свое существование с маркетинговой идеи, которая передается в проектный отдел. Проектный отдел моделирует функции продукта и подготавливает проект. Отдел производства изучает и корректирует проект, приводя его в соответствие с имеющимся оборудованием. Проектный отдел принимает изменения, после чего проект изучает служба поддержки: хороший проект должен подразумевать удобство ремонта. Проектный отдел принимает предложения службы поддержки и проверяет, что после корректировок проект удовлетворяет требованиям, предъявленным к целевой функциональности.

11. Разработайте диаграмму деятельности покупки товара в автомате по продаже продуктов питания.

12. Разработайте диаграмму деятельности покупки товара в автомате по продаже продуктов питания.

13. Разработайте диаграмму деятельности покупки авиабилета на сайте авиакомпании.

14. Разработайте диаграмму деятельности покупки железнодорожного билета на сайте <https://www.rzd.ru>.

15. Разработайте диаграмму деятельности продажи товара на сайте <https://www.avito.ru>

Контрольные вопросы

1. В чем состоит назначение диаграммы деятельности?
2. Чем диаграммы деятельности отличаются от блок-схем? Какие преимущества это дает разработчикам?
3. Какова роль диаграмм деятельности в проектировании информационных систем?
4. Что описывает состояние деятельности на диаграмме деятельности?
5. В чем сходство и в чем отличия диаграмм состояний и деятельности?
6. С какими схемами, используемыми в структурном программировании, можно сравнить диаграмму деятельности? Что у них общего и в чем отличия?
7. Каким образом на диаграмме деятельности отображается разветвление процесса?
8. Для чего на диаграмме деятельности используется элемент «Дорожка»?
9. Каковы правила и рекомендации при построении диаграммы деятельности?
10. Как моделируются начальное и конечное состояния?
11. Как обозначают на диаграмме точку принятия решения? Когда нужно моделировать принятия решений?

6. Диаграммы компонентов

Цель работы – сформировать навык построения диаграммы компонентов в Visual Paradigm.

Задачи:

- ознакомиться с назначением диаграммы компонентов и ее элементами.
- изучить возможности Visual Paradigm для построения диаграммы компонентов.
- построить диаграмму компонентов в Visual Paradigm в соответствии с предложенными заданиями.

Используемое ПО: Visual Paradigm, Draw.io, Rational Rose.

Краткая теория

Диаграммы компонентов используются для моделирования статического вида системы с точки зрения реализации. Этот вид диаграмм в первую очередь связан с управлением конфигурацией частей системы, составленной из компонентов, которые можно соединять между собой различными способами.

Диаграмма компонентов (component diagram) описывает особенности физического представления разрабатываемой системы, позволяя определить ее архитектуру, установив зависимости между программными компонентами, в роли которых могут выступать исходный, бинарный и исполняемый коды. Данная диаграмма обеспечивает согласованный переход от логического к физическому представлению системы в виде программных компонентов.

Диаграмма компонентов разрабатывается для следующих целей:

- визуализация общей структуры исходного кода программной системы;
- спецификация исполнимого варианта программной системы.
- обеспечение многократного использования отдельных фрагментов программного кода.
- представление концептуальной и физической схем баз данных.

Компонент (component) - элемент модели, представляющий некоторую модульную часть системы с инкапсулированным содержимым, спецификация которого является взаимозаменяемой в его окружении. На рис. 54 представлен пример изображения компонентов.

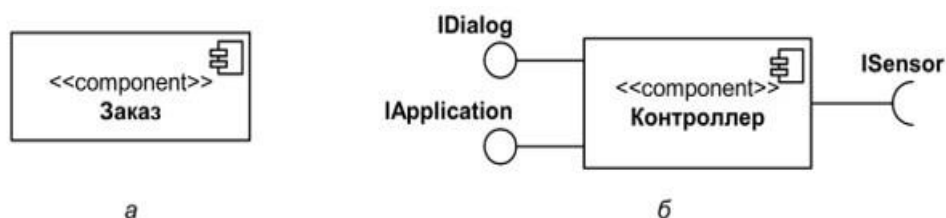


Рисунок 54. Примеры изображения: а – простого компонента, б – компонента с интерфейсами

Предоставляемый интерфейс (provided interface) - интерфейс, который компонент предлагает для своего окружения. Требуемый интерфейс (required interface) - интерфейс, который необходим компоненту от своего окружения для выполнения заявленной функциональности, контракта или поведения.

Предоставляемые и требуемые интерфейсы в языке UML 2.0 изображаются с использованием специальной нотации, которая получила название "шара и шарнира" или "леденца на палочке". Эта нотация позволяет компактно изобразить графическое соединение компонентов и удовлетворяет требованию масштабирования сложных систем. Предоставляемые интерфейсы компонента обозначаются линией с шаром, а требуемые интерфейсы - линией с гнездом. На рис. 55 представлен пример изображения компонентов с интерфейсами.

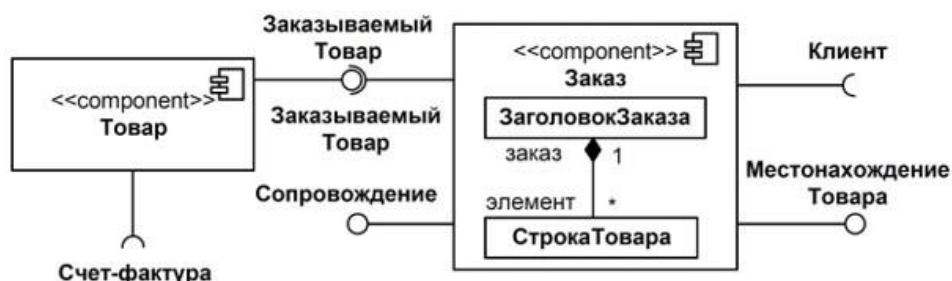


Рисунок 55. Пример изображения компонентов с интерфейсами

Порт определяет различимую точку взаимодействия между компонентом и окружающей его средой или между компонентом и его внутренними частями. Изображаются порты так же, как и на других диаграммах структуры - в виде небольших квадратов, расположенных на границе символа компонента.

На диаграмме компонентов также могут быть представлены отношения зависимости между компонентами. Это представление часто называют обзорным, поскольку на нем не изображаются интерфейсы и порты (рис. 56 и 57).

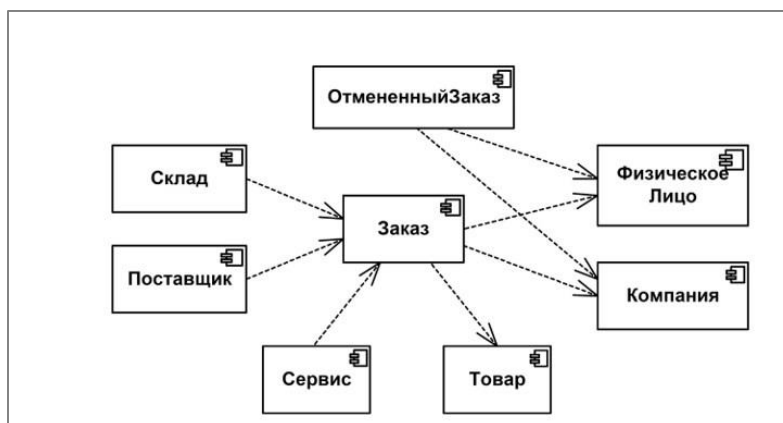


Рисунок 56. Пример отношений зависимости между компонентами

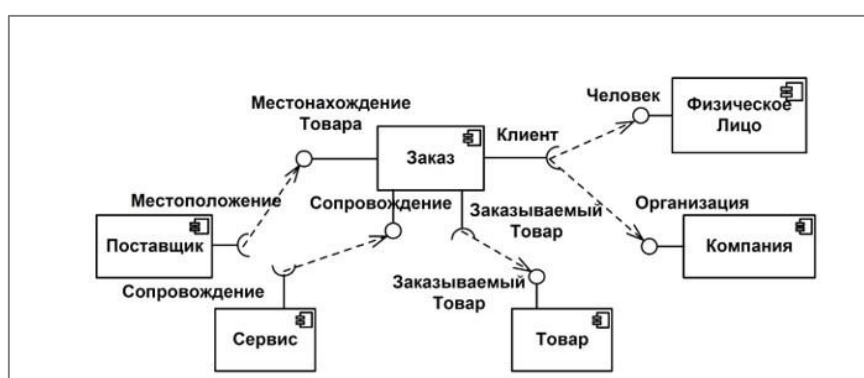


Рисунок 57. Отношения зависимости на диаграмме компонентов с интерфейсами

Реализация (realization) - специализация отношения зависимости для связи компонентов с классификаторами, которые реализуют функциональность этого компонента (рис. 58).

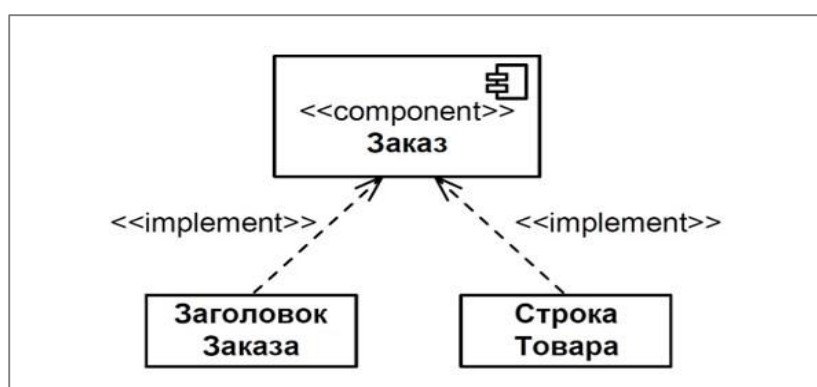


Рисунок 58. Пример отношения реализации компонента

Компонент, как элемент модели, может иметь различную физическую реализацию. Для более точной спецификации формы реализации компонента

можно использовать текстовый стереотип, который указывается выше строки с именем компонента. Компоненты могут иметь следующие стандартные стереотипы:

- «*file*» – любой файл кроме таблицы;
- «*executable*» – программа (исполняемый файл);
- «*library*» – статическая или динамическая библиотека;
- «*document*» – остальные файлы (например, файл справки);
- «*table*» – таблица базы данных.

Этапы реализации:

Шаг 1. Запустить программу Visual Paradigm

Шаг 2. Выбрать вкладку System Design

Шаг 3. Выбрать Component Diagram и нажать Next (рис. 59)

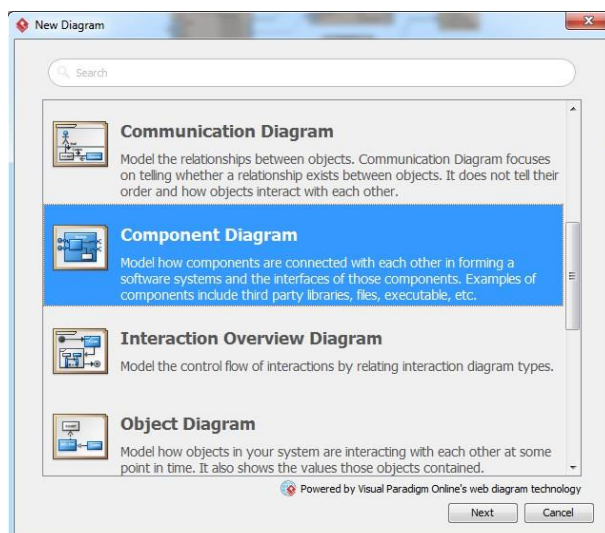


Рисунок 59. Главная форма программы

Шаг 4. Выбрать шаблон «Blank», нажать Next.

Пример диаграммы компонентов для поликлиники

В данном случае (рис. 60), в зависимости от роли пользователя осуществляется вызов одной из подсистем: АРМ сотрудника регистратуры или АРМ врача. Сотрудник регистратуры работает с подсистемой, обрабатывающей информацию из двух баз данных: БД врачей и БД услуг. Кроме того, на диаграмме показано, что терминал для выписывания талонов на прием является интерфейсом.

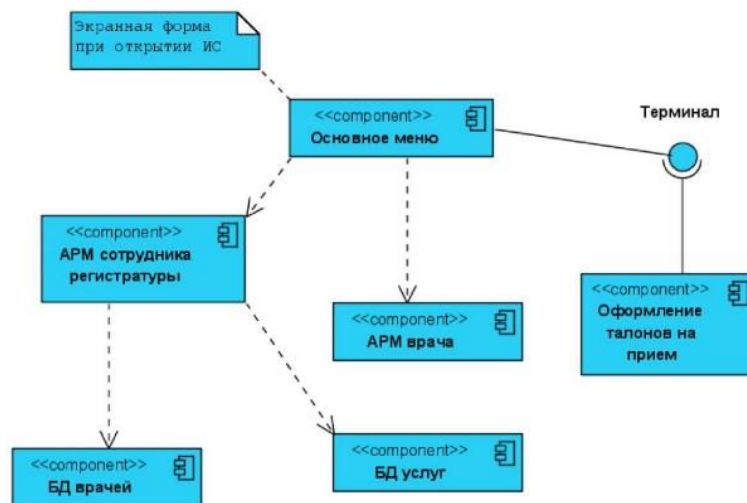


Рисунок 60. Диаграмма компонентов для поликлиники

Создайте стандартный компонент, выбрав в панели элементов объект Component. Определите созданному элементу имя «БД врачей». Для построения диаграммы, можно установить данному компоненту стереотип «table». Сделать это можно выбором команды Stereotypes – table контекстного меню объекта. В результате, компонент «БД врачей» приобретает следующий вид (рис. 61).

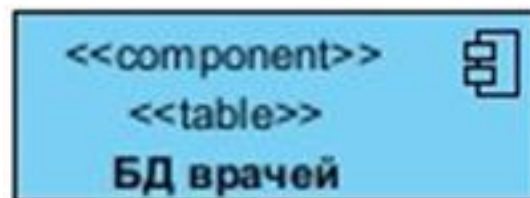


Рисунок 61. Компонент «БД врачей»

Аналогичным образом добавьте в диаграмму компонент «БД услуг», также определив элементу стереотип «table» (рис. 62).

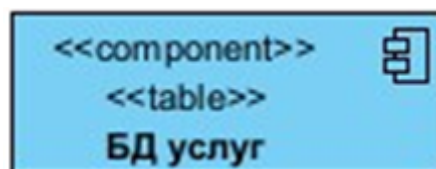


Рисунок 62. Компонент «БД услуг»

Далее, создайте компоненты «АРМ сотрудника регистратуры» и «АРМ врача». На программном уровне данные компоненты представляют собой подсистемы, которые пользователь, в зависимости от своего служебного

статуса, через главный интерфейс вызывает на исполнение. Задайте компонентам стереотип «subsystem».

Еще один элемент диаграммы - спецификация задачи (исполняемый файл) «Основное меню» – создается аналогично предыдущим, но имеет стереотип Executable (Исполнимый). Создайте элемент «Оформление талонов на приём». Так как талоны на приём реализуются в форме документа, компонент должен иметь стереотип «document». Последний компонент – интерфейс «Терминал». Для создания интерфейса выберите в панели элементов объект Interface; вторым щелчком добавьте его в область построения. Имя интерфейсу определяется с заглавной латинской буквы «I». Интерфейс должен быть связан с реализующим его компонентом посредством обобщения (Generalization) и ассоциацией (Association) с компонентом, который использует его в процессе выполнения. В нашем случае интерфейс «Терминал» реализуется компонентом «Основное меню», а используется элементом системы «Оформление талонов на приём» (рисунок 6). Наконец, в соответствии с рисунком 6, определите зависимости между остальными компонентами – кнопка «Dependency» панели элементов.

Правила и рекомендации при построении диаграммы деятельности

1. Перед разработкой диаграмм компонентов необходимо решить, из каких физических частей (файлов) будет состоять программная система.

2. При спецификации общей структуры исходного кода системы необходимо учитывать специфику языка программирования, с помощью которого реализуются компоненты.

3. Для спецификации исполнимого варианта системы необходимо иметь в наличии предварительную топологию системы, т. е. набросок диаграммы развертывания. Для каждого узла в сети может быть построена диаграмма компонентов, определяющая набор файлов, необходимых для работы подсистемы (подсистем) на отдельном рабочем месте.

4. На диаграмме могут быть представлены отношения зависимости между компонентами и включенными в них классами. Эта информация имеет важное значение для обеспечения согласованности между логическим и физическим представлениями системы. В этом случае зависимость можно показать двумя способами: классы показать отдельно от компонента и связать компонент с каждым классом отношением зависимости; классы отобразить внутри символа компонента.

5. Рекомендуется минимизировать количество связей между исполняемыми компонентами, а необходимое взаимодействие осуществлять через хранимую в единой базе данных информацию.

6. Для наглядного отображения специфики компонентов можно вместо стандартного символа компонента со строковым стереотипом внутри использовать графические стереотипы.

Задания для самостоятельного выполнения

1. Разработайте диаграмму компонентов для торгового предприятия.
2. Разработайте диаграмму компонентов для работы автохозяйства.
3. Разработайте диаграмму компонентов для работы аптеки.
4. Разработайте диаграмму компонентов для газозаправочной станции.
5. Разработайте диаграмму компонентов для кредитных операций юридических лиц.
6. Разработайте диаграмму компонентов для организации оптового бизнеса.
7. Разработайте диаграмму компонентов для организации гостиничного бизнеса.
8. Разработайте диаграмму компонентов для организации авторемонтного бизнеса.
9. Разработайте диаграмму компонентов для организации продаж автомобилей в автосалоне.
10. Разработайте диаграмму компонентов для организации ресторанного бизнеса.
11. Разработайте диаграмму компонентов для организации делопроизводства.
12. Разработайте диаграмму компонентов для организации системы склад-магазин.
13. Разработайте диаграмму компонентов для работы рекламного бизнеса.
14. Разработайте диаграмму компонентов для организации сбытовой деятельности.
15. Разработайте диаграмму компонентов для организации кадровой службы предприятия.

Контрольные вопросы

1. В чем состоит назначение диаграммы компонентов?
2. Как создать отношения между компонентами диаграммы?

3. Перечислите основные элементы диаграммы компонентов.
4. Что такое компонент?
5. Какие виды компонентов существуют?
6. Что такое интерфейс?
7. Как изображается интерфейс на диаграмме компонентов?
8. Какие виды и стереотипы определены для компонентов?
9. Для чего на диаграмме компонентов используются зависимости?

7. Диаграммы развертывания

Цель работы – сформировать навык построения диаграммы развертывания в Visual Paradigm.

Задачи:

- ознакомиться с назначением диаграммы развертывания и ее элементами;
- изучить возможности Visual Paradigm для построения диаграммы развертывания;
- построить диаграмму развертывания в Visual Paradigm в соответствии с предложенными заданиями.

Используемое ПО: Visual Paradigm, Draw.io, Rational Rose.

Краткая теория

При создании распределенных или клиент-серверных приложений требуется визуализировать сетевую инфраструктуру программной системы.

Диаграмма развертывания (deployment diagram) предназначена для представления общей конфигурации или топологии распределения программной системы и содержит изображение размещения различных артефактов (исполняемых компонентов и динамических библиотек) по отдельным узлам системы. Она визуализирует только те элементы физического представления модели, которые существуют во время выполнения или исполнения программной системы, например, исполняемые компоненты. Те элементы, которые не используются на этапе выполнения на диаграмме развертывания, как правило, не показываются. Так, например, компоненты с исходными текстами программ могут присутствовать на диаграмме компонентов, а на диаграмме развертывания они не указываются, а может указываться только исполняемый компонент, получаемый в результате их компиляции.

Следует отметить, что в отличие от диаграмм логического представления и диаграмм компонентов, диаграмма развертывания проектируется, как правило, в единственном экземпляре, так как она должна всецело отражать особенности топологии и реализации разрабатываемой системы.

На диаграммах развертывания визуализируется конфигурация обрабатывающих узлов, на которых выполняется программная система, и компонентов, размещенных в этих узлах, а также специфицируются взаимосвязи между отдельными узлами в виде отношений ассоциации и зависимости.

При разработке диаграмм развертывания преследуются следующие цели:

специфицировать физические узлы; показать физические связи между узлами; выявить узкие места.

Узел (node) является элементом модели, который представляет некоторый вычислительный ресурс для развертывания на нем различных артефактов (рис. 63).



Рисунок 63. Графическое изображение: а - узла в качестве типа; б - узла в качестве экземпляра

Среда выполнения (execution environment) представляет собой узел, который обладает функциональностью, необходимой для практического выполнения развернутых на нем исполнимых артефактов (рис. 64).

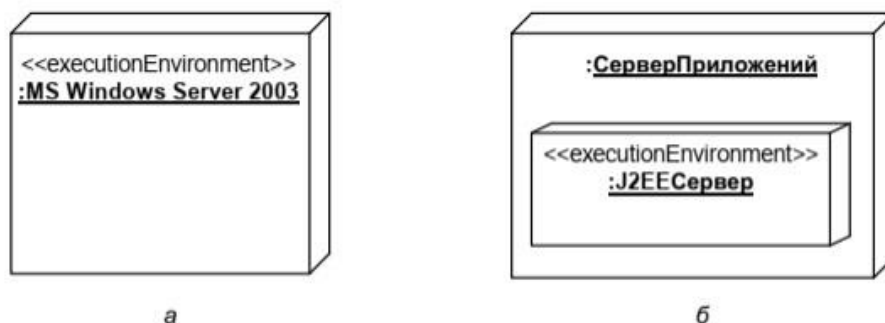
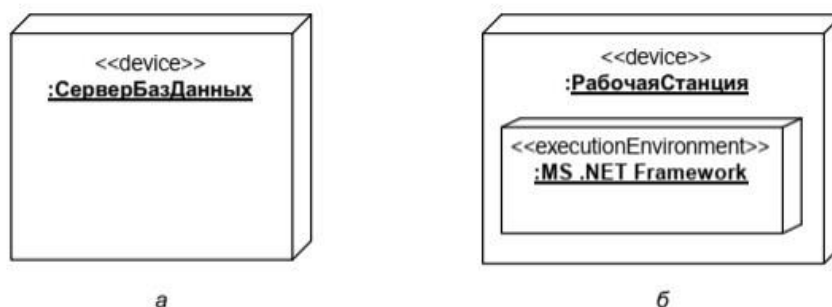


Рисунок 64. Примеры изображения: а – среды выполнения; б – среды выполнения в узле

Устройство (device) представляет собой узел, который обладает некоторым общим вычислительным ресурсом со способностью обрабатывать развернутые на нем артефакты (рис. 65).



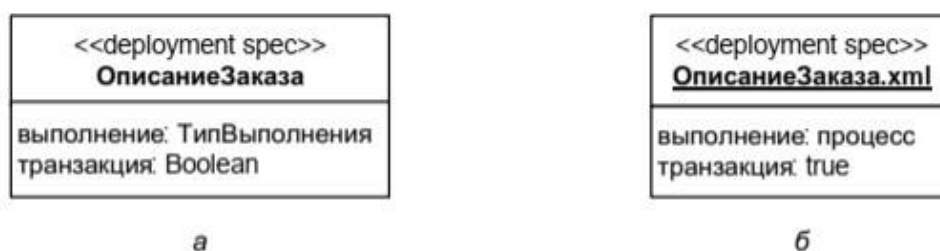
*Рисунок 65. Примеры изображения: а – простого устройства;
б – устройства контейнера*

Артефакт (artifact) представляет собой элемент модели, который специфицирует некоторую физически существующую часть информации, используемую или производимую в ходе разработки программного обеспечения или в процессе развертывания и функционирования системы (рис. 66).



*Рисунок 66. Примеры изображения: а – артефакта в качестве типа;
б – экземпляра артефакта*

Спецификация развертывания (deployment specification) описывает множество свойств, которые определяют параметры выполнения артефакта компонента, развертываемого на некотором узле (рис. 67).



*Рисунок 67. Примеры изображения спецификации развертывания
для артефакта: а – уровня типа; б – уровня экземпляра*

Развертывание (deployment) представляет собой размещение артефакта или экземпляра артефакта на некоторой цели развертывания (рис. 68 и рис. 69).

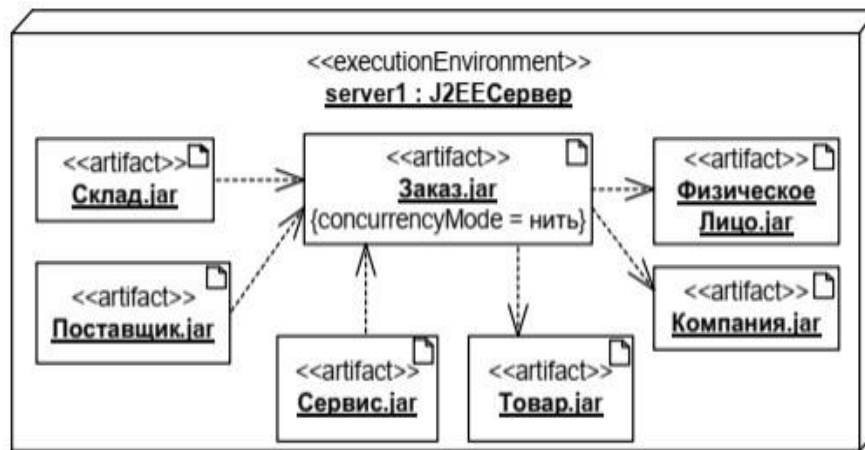


Рисунок 68. Первый способ визуального представления множества экземпляров артефактов, развернутых на узле

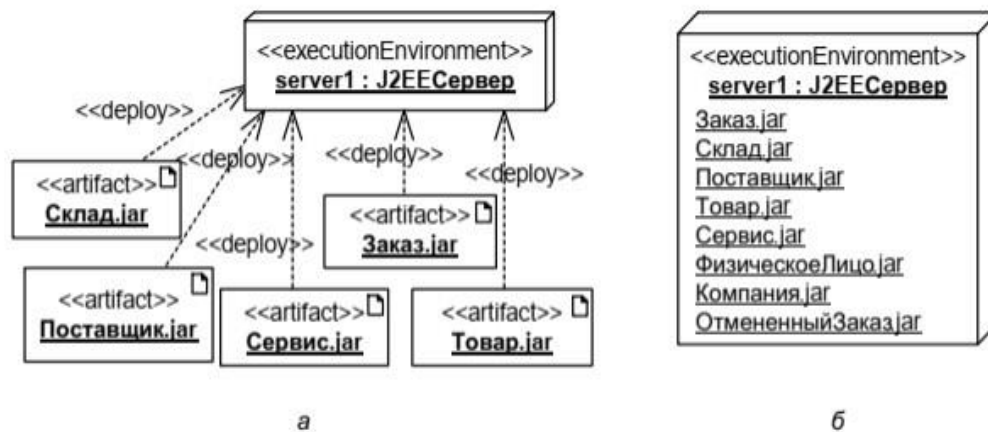


Рисунок 69. Второй (а) и третий (б) способы визуального представления множества экземпляров артефактов, развернутых на узле

Манифестация (manifestation) представляет собой отношение для спецификации конкретного физического воплощения одного или нескольких элементов модели посредством артефакта (рис. 70).

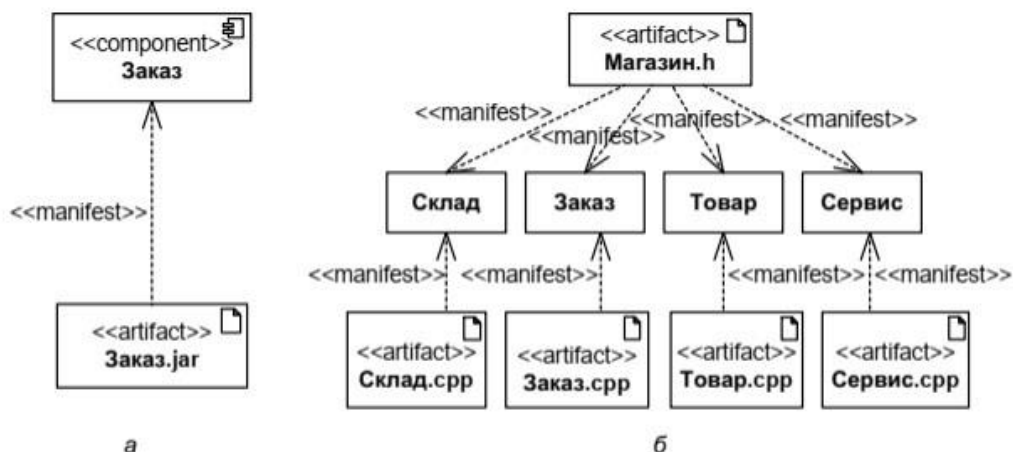


Рисунок 70. Примеры изображения отношения манифестации: а – между артефактов и компонентом; б – между артефактами, компонентами и классами

Путь коммуникации (communication path) является ассоциацией между двумя целями развертывания, посредством которой они обладают способностью обмениваться сигналами и сообщениями (рис. 71).

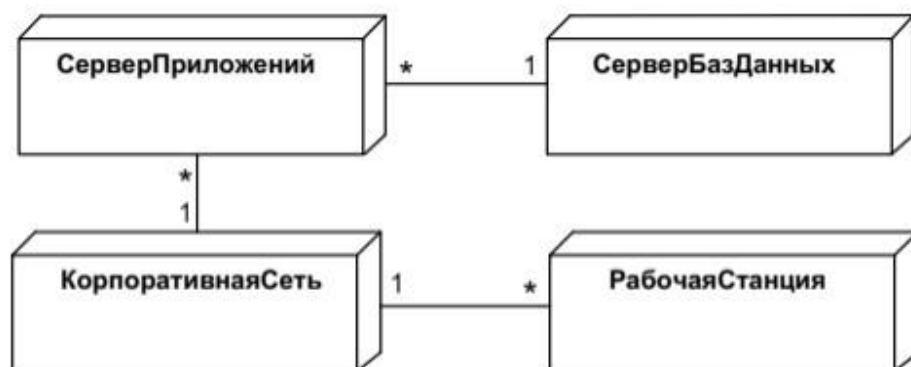


Рисунок 71. Пути коммуникации между типами узлов

Этапы реализации:

Шаг 1. Запустить программу Visual Paradigm

Шаг 2. Выбрать вкладку System Design

Шаг 3. Выбрать Deployment Diagram и нажать Next (рис. 72).

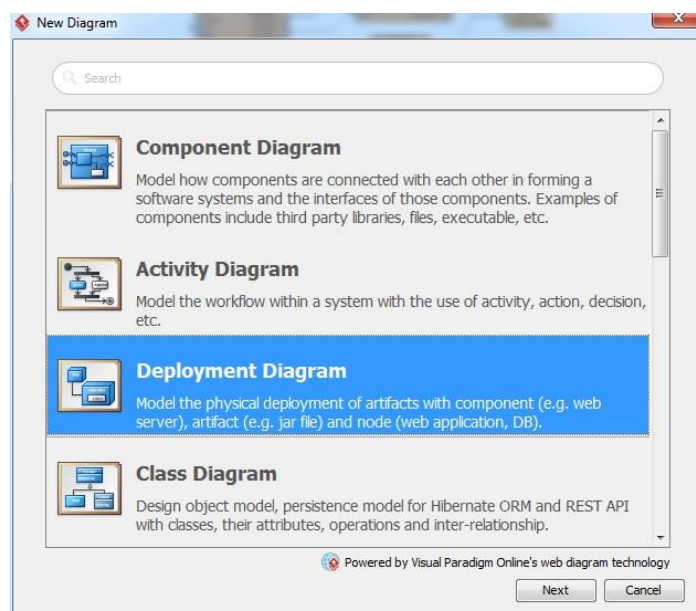


Рисунок 72. Выбор типа диаграммы

Шаг 4. Выбрать «Blank», нажать Next.

Шаг 5. Задать имя диаграмме и нажать ОК.

Пример построения диаграмма размещения для информационной системы поликлиники

В данном примере (рис. 73) система поликлиники состоит из АРМ сотрудника регистратуры, АРМ врачей и терминала для выдачи талонов на посещение, то есть из отдельных физических устройств, или узлов (node). Диаграмма развертывания используется менеджером проекта, пользователями, архитектором системы и эксплуатационным персоналом, чтобы понять физическое размещение системы и расположение ее отдельных подсистем.



Рисунок 73. Диаграмма развертывания для системы «Поликлиника»

Добавьте в диаграмму узел, соответствующий региональному серверу системы. Для этого, выберите в панели элементов объект Node, затем щелкните мышью на области построения. Определите узлу имя «Сервер поликлиники». Далее, аналогичным образом и согласно рис. 73, добавьте в область построения узлы: «АРМ сотрудника регистратуры» и «АРМ врача». В рассматриваемой диаграмме размещения также существует компонент «Электронный терминал». Данный компонент является функционирующим в рамках системы устройством или девайсом. Для добавления в диаграмму узла устройства, выберите в панели элементов (выпадающий список элемента Node) объект Device Node после чего, нажатием мыши добавьте объект в область построения. Для детального описания свойств или особенностей узлов диаграммы размещения используют примечания (объект Note панели элементов). Примечания не являются частью системы, т.е. не соответствуют какому-либо физическому объекту, а служат лишь для описания её компонент на диаграмме. Добавьте необходимые примечания для узлов рассматриваемой диаграммы, как показано на рис. 73. Определите необходимые ассоциации между компонентами системы

(объектами диаграммы). Ассоциацию компонент системы с элементами-примечаниями можно установить, выбрав тип связи Generic Connector (кнопка Generic Connector) панели элементов программы.

Правила и рекомендации при построении диаграммы развертывания

1. Перед разработкой диаграммы развертывания необходимо идентифицировать: категории (типы) пользователей; аппаратные, технические и другие типы устройств; виды и требуемую пропускную способность каналов связи.

2. Должны быть рассмотрены варианты прокладки новой или модернизации существующей корпоративной сети организации.

3. В целях наглядного представления распределенной информационной системы на диаграмме рекомендуется отображать компоненты, интерфейсы и связи между ними.

4. Для наглядного отображения специфики узлов вместо стандартного символа узла со строковым стереотипом могут использоваться графические стереотипы.

Задания для самостоятельного выполнения

1. Разработайте диаграмму развертывания для организации кадровой службы предприятия.

2. Разработайте диаграмму развертывания для организации проката автомобилей.

3. Разработайте диаграмму развертывания для организации работы автошколы.

4. Разработайте диаграмму развертывания для организации работы диспетчерской службы автовокзала.

5. Разработайте диаграмму развертывания для организации работы аварийной диспетчерской службы ЖКХ.

6. Разработайте диаграмму развертывания для организации питания в частном детском саду

7. Разработайте диаграмму развертывания для организации работы приюта для животных.

8. Разработайте диаграмму развертывания для организации работы бюро находок.

9. Разработайте диаграмму развертывания для организации банковского бизнеса (на примере кредитных операций физических лиц).

10. Разработайте диаграмму развертывания для организации работы кружков образовательного учреждения.

11. Разработайте диаграмму развертывания для организации работы ресторана по доставке блюд.

12. Разработайте диаграмму развертывания для организации продажи билетов в кинотеатр.

13. Разработайте диаграмму развертывания для организации составления расписания спектаклей кукольного театра.

14. Разработайте диаграмму развертывания для организации работы службы доставки офисной мебели.

15. Разработайте диаграмму развертывания для организации работы платных курсов в образовательном учреждении.

Контрольные вопросы

1. Какова роль диаграмм развертывания в проектировании информационных систем?
2. Какие группы специалистов участвуют в разработке диаграмм развертывания?
3. Является ли диаграмма развертывания единой для системы в целом или может ли она описывать какую-то ее часть?
4. В каких случаях использование диаграммы развертывания нецелесообразно?
5. Что собой в языке UML представляет узел?
6. Какие виды узлов используются в UML?
7. В чем разница между элементами узел и «Экземпляр узла»?
8. Для чего на диаграмме развертывания могут быть представлены компоненты? Какие виды компонентов для этого используются?
9. Какую роль на диаграмме развертывания играют интерфейсы?
10. Для чего на диаграмме развертывания используются сообщения и зависимости?

Список источников и литературы

1. Белов, В.В. Проектирование информационных систем: Учебник / В.В. Белов. - М.: Академия, 2018. - 144 с.
2. Гвоздева, Т.В. Проектирование информационных систем: технология автоматизированного проектирования. Лабораторный практикум. Учебно-справочное пособие / Т.В. Гвоздева, Б.А. Баллод. - СПб.: Лань, 2018. - 156 с.
3. Гвоздева, Т.В. Проектирование информационных систем. Стандартизация: Учебное пособие / Т.В. Гвоздева, Б.А. Баллод. - СПб.: Лань, 2019. - 252 с.
4. Орлов С.А., Цилькер Б.Я. Технологии разработки программного обеспечения. СПб.: Питер, 2012. — 608 с.: ил.
5. Процессный подход к управлению. Моделирование бизнес-процессов/ В.Г. Елиферов., В.В. Репин — М.: Манн, Иванов и Фербер, 2013. — 544 с.: ил. ISBN 978-5-91657-554-5
6. Г. Поллис, Л. Огастин, К. Лоу, Д. Мадхар Разработка программных проектов на основе RUP. М.: Бином, 2011. — 256 с.
7. Современные методы и средства проектирования информационных систем [Электронный ресурс]: учебное пособие / сост. Деменков М.Е., Деменкова Е.А. — Электрон. дан. — Архангельск: САФУ, 2015. — 90 с. — Режим доступа: <https://e.lanbook.com/book/96547>.
8. Свод знаний по управлению бизнес-процессами: BPM СВОК 3.0 [Электронный ресурс] / под ред. А. А. Белайчука, В. Г. Елифёрова. — Электрон. дан. — Москва: Альпина Паблицер, 2016. — 480 с.